HOSPITAL MANAGEMENT SYSTEM DATABASE

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

B. Nandha Kishore – 25B11AI110

V. Ram Nikhil – 25B11AIC32

K. Yaswanth Reddy – 25B11AI525

G. Mahesh – 25B11AI335

Under the esteemed guidance of

Mr. T.Srinivasulu M.Tech, (Ph.D.)

Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “HOSPITAL MANAGEMENT SYSTEM DATABASE” is being submitted by B. Nandha Kishore (Roll No. 25B11AI110), V. Ram Nikhil (Roll No. 25B11AIC32), K. Yaswanth Reddy (Roll No. 25B11AI525) and G. Mahesh (Roll No. 25B11AI335) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

Mr. T. Srinivasulu, M. Tech, (Ph.D.)

Associate professor

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.

Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “HOSPITAL MANAGEMENT SYSTEM DATABASE” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

B. Nandha Kishore (25B11AI110)

V. Ram Nikhil (25B11AIC32)

K. Yaswanth Reddy (25B11AI525)

G. Mahesh (25B11AI335)

Date:

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr. K. Naga Bhargavi, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

B. Nandha Kishore	25B11AI110
V. Ram Nikhil	25B11AIC32
K. Yaswanth Reddy	25B11AI525
G. Mahesh	25B11AI335

ABSTRACT

Hospitals generate large volumes of interrelated data covering patients, doctors, appointments, prescriptions, billing, wards and staff. Managing this data with manual records or spreadsheets leads to duplicate entries, scheduling conflicts and no cross-referencing between departments. This project designs and implements a centralized relational database for a Hospital Management System that enforces data integrity through keys and supports fast, structured retrieval through SQL. The database is built around eight entities — Patient, Doctor, Appointment, Prescription, Billing, Ward, Staff and Admission — normalized to 3NF and implemented as eight relational tables linked through nine foreign-key constraints. MySQL is the database server, MySQL Workbench is used for query execution, draw.io for the ER and schema diagrams, and the project files are maintained on GitHub. The implementation covers table creation with primary and foreign key constraints, ten sample records per table, and twenty-four SQL operations including joins, aggregate functions with GROUP BY and HAVING, nested and correlated subqueries, views, set operations, and DCL/TCL commands, each supported by execution screenshots. The result is a reliable, working foundation for core hospital operations that can be extended with reporting or additional modules.

INDEX

Chapter	Title	Page No.
	Certificate	2
	Declaration	3
	Acknowledgement	4
	Abstract	5
	List of Figures	8
	List of Tables	9
1	**Introduction**	11
	1.1 Introduction to the Domain	11
	1.2 Problem Statement	11
	1.3 Motivation and Need for a Database	12
	1.4 Objectives of the Project	12
	1.5 Scope of the Project	13
	1.6 Organization of the Report	13
2	**System Study and Tools**	14
	2.1 Existing System and Its Limitations	14
	2.2 Proposed Database System	14
	2.3 Tools and Technologies Used	15
	2.4 Software and Hardware Requirements	15
3	**Database Design (Review – I)**	15
	3.1 Identified Entities	15
	3.2 Entity Attributes and Attribute Types	16
	3.3 ER Diagram	19
	3.4 Relationships and Cardinality	20
	3.5 Relational Database Schema	21
	3.6 Primary Keys and Foreign Keys	26
	3.7 Normalization up to 3NF	26
4	**Database Implementation (Review – II)**	29

	4.1 Database and Table Creation – DDL	29
	4.2 Sample Data Insertion – DML	33
	4.3 Constraints and Data Integrity	40
	4.4 Data Validation Rules	41
5	**SQL Operations and Queries**	42
	5.1 DQL – Basic Retrieval Queries	42
	5.2 Join Operations	42
	5.3 Aggregate Functions, GROUP BY and HAVING	43
	5.4 Nested and Correlated Subqueries	43
	5.5 Views and Relational Set Operations	44
	5.6 DCL and TCL Operations	44
6	**Modules and Functionalities**	46
7	**Results – Sample Outputs and Screenshots**	48
8	**Testing and Data Validation**	58
9	**GitHub Repository and Team Contribution**	59
10	**Conclusion and Future Enhancements**	60
	References	61
	Appendix A – Complete SQL Script	62

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
3.1	ER Diagram of the Hospital Management System	20
3.2	Relational Schema Diagram	26
4.1	Successful Creation of Tables	33
4.2	Sample Data Inserted into Hospital Tables	40
4.3	Referential Integrity Enforced by the Database	41
7.1	Patient records retrieved from the Patient table.	49
7.2	Female patients above 30 years retrieved using WHERE.	49
7.3	Patients younger than 30 or older than 50 retrieved using OR.	49
7.4	Available doctors retrieved using WHERE.	49
7.5	Patient records ordered from youngest to oldest.	50
7.6	Distinct blood groups retrieved from the Patient table.	50
7.7	Appointment details with patient and doctor names using INNER JOIN.	50
7.8	Patient records with appointment information using LEFT OUTER JOIN.	51
7.9	Doctor records with appointment information using RIGHT OUTER JOIN.	51
7.10	Patient and billing records using a FULL OUTER JOIN simulation.	52
7.11	Total treatment amount calculated using SUM.	52
7.12	Patient count grouped by gender using COUNT and GROUP BY.	53
7.13	Average bill grouped by payment status using HAVING.	53
7.14	Patients older than the average age using a nested subquery.	53
7.15	Doctors with completed appointments using a correlated subquery.	54
7.16	Available doctors view created as an updatable view.	54

7.17	Patient billing summary created as a non-updatable view.	54
7.18	Patient IDs from appointments and billing combined using UNION.	55
7.19	Patients in both appointments and billing using the INTERSECT equivalent.	55
7.20	Patients in appointments but not billing using the EXCEPT equivalent.	55
7.21	Read access granted to a reporting user using GRANT.	56
7.22	SELECT privilege removed from the reporting user using REVOKE.	56
7.23	Appointment status change permanently saved using COMMIT.	56
7.24	Billing change undone after a savepoint using ROLLBACK.	57

LIST OF TABLES

Table No.	Title of the Table	Page No.
2.1	Tools and Technologies Used	15
2.2	Software and Hardware Requirements	15
3.1	Entities Identified for the Selected Domain	16
3.2	Attributes of Each Entity	16
3.3	Relationships and Cardinality	20
3.4	Relational Database Schema	21
3.5	Primary Key and Foreign Key Definitions	26
3.6	Normalization Steps	27
4.1	Sample Data Insertion Summary	33
4.2	Constraints and Data Integrity Rules	40
5.1	Basic Retrieval Queries	42
5.2	Join Operations	42
5.3	Aggregate, GROUP BY and HAVING Queries	43

5.4	Subqueries	43
5.5	Views and Set Operations	44
5.6	DCL and TCL Operations	44
6.1	Modules and Their Functionalities	46
8.1	Test Cases and Results	58
9.1	Team Responsibilities	59
9.2	Review-wise Progress Record	59

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

The domain selected for this capstone project is Hospital Management. A hospital, whether a small nursing home or a large multi-department facility, handles large volumes of interrelated data on a daily basis, covering patient records, doctor details, appointment scheduling, prescriptions, billing and ward or bed allocation. Every patient visit generates information that must be linked correctly across departments: the doctor consulted, the prescription issued, the ward occupied in case of admission, and the resulting bill.

Because this information is naturally relational — patients are linked to appointments, appointments to doctors and prescriptions, admissions to wards and billing — it is well suited to being modeled and managed through a relational database rather than through flat files or manual registers. This project focuses on designing and building such a database to support the core day-to-day data-handling needs of a hospital.

1.2 Problem Statement

Hospitals often rely on manual records or spreadsheets to manage patient, doctor and billing data. This approach leads to data redundancy and inconsistency, such as duplicate patient entries, because there is no centralized access and records are scattered across departments. Manual scheduling frequently results in double-booked appointments, retrieval of records — especially a patient's history — is slow, and there are no data integrity checks to prevent invalid or incomplete entries. Generating reports such as billing summaries or ward occupancy is also difficult under this approach.

This project addresses these problems by designing a centralized relational database to store and manage hospital data. The database enforces data integrity through keys and constraints, enables fast and structured retrieval through SQL queries, supports concurrent access for multiple departments such as reception, doctors and billing, and provides a foundation for reliable reporting and analytics.

1.3 Motivation and Need for a Database

A DBMS is required for the hospital domain, rather than files or spreadsheets, for several reasons. Data integrity is enforced through constraints that prevent duplicate or invalid records, which is critical when dealing with patient and billing data. Retrieval is efficient, since instant SQL queries replace slow manual searching through registers. Hospital data is also naturally relational — patients, doctors, appointments and bills are interlinked — and a DBMS models these relationships directly instead of duplicating data across separate files. A DBMS additionally supports concurrent access, so multiple staff members can safely access and update data at the same time, and it enables role-based security so that sensitive patient and billing data is accessible only to authorized users.

1.4 Objectives of the Project

The objectives of the project are as follows.

1. To design a normalized relational database for the selected domain with at least six entities.
2. To implement the database with appropriate primary keys, foreign keys and integrity constraints.
3. To enable efficient storage, retrieval and management of patient and hospital records.
4. To support key operations such as appointment scheduling, prescription tracking and billing through SQL queries.
5. To provide a scalable foundation that can be extended with reporting or additional modules in the future.

1.5 Scope of the Project

The database covers the core data-handling needs of a single hospital, including patient registration, doctor and staff records, appointment scheduling, prescription tracking, billing, and ward and admission management. It stores and manages the eight entities identified in this report — Patient, Doctor, Appointment, Prescription, Billing, Ward, Staff and Admission — and supports SQL-based retrieval, reporting and integrity enforcement across these entities.

The project does not cover a front-end user interface, integration with external systems such as insurance or pharmacy networks, real-time notifications, or multi-hospital / multi-branch data management. These are identified as possible future enhancements in Chapter 10.

1.6 Organization of the Report

This report is organized into ten chapters. Chapter 2 presents the system study and the tools used. Chapter 3 covers the complete database design including the ER diagram, relational schema and normalization. Chapter 4 describes the database implementation with DDL and sample data. Chapter 5 presents the SQL operations and queries executed. Chapter 6 explains the modules and functionalities. Chapter 7 presents the results with sample outputs. Chapter 8 covers testing and data validation. Chapter 9 details the GitHub repository and the contribution of each team member. Chapter 10 concludes the report and outlines future enhancements.

CHAPTER 2: SYSTEM STUDY AND TOOLS

2.1 Existing System and Its Limitations

Without a proper database, a hospital typically manages patient, doctor and billing information through paper registers or independent spreadsheets maintained by different departments. This existing approach suffers from several limitations: data redundancy and inconsistency, such as the same patient being registered more than once with slightly different details; no centralized access, since records are scattered across reception, doctors' rooms and the billing counter; manual, error-prone appointment scheduling that can result in double-booking; slow retrieval of records, particularly a patient's full history; the absence of data integrity checks, allowing invalid or incomplete entries; and difficulty in generating consolidated reports such as billing summaries or ward occupancy figures.

2.2 Proposed Database System

The proposed system is a centralized relational database, built around eight normalized tables — Patient, Doctor, Appointment, Prescription, Billing, Ward, Staff and Admission — that store and interlink all core hospital data. Over the existing manual/spreadsheet approach, it offers enforced data integrity through primary and foreign key constraints, elimination of redundant or duplicate records, fast structured retrieval through SQL queries instead of manual searching, safe concurrent access for multiple departments at once, and a reliable foundation for generating reports and analytics on demand.

2.3 Tools and Technologies Used

Table 2.1: Tools and Technologies Used

Category	Tool / Technology Used	Version / Remarks
Database Server	MySQL	8.0
Query / Admin Tool	MySQL Workbench	8.0 CE
ER Diagram Tool	draw.io (diagrams.net)	Web-based
Version Control	GitHub	—
Documentation	Microsoft Word	365

The project uses MySQL Server 8.0 as the relational database engine, MySQL Workbench 8.0 CE for executing and verifying SQL, draw.io for ER and relational schema diagrams, GitHub for source control, and Microsoft Word for the project report.

2.4 Software and Hardware Requirements

Table 2.2: Software and Hardware Requirements

Requirement Type	Specification
Operating System	Windows 10 / 11 (or equivalent)
Database Software	MySQL Server 8.0 with MySQL Workbench
Processor	Intel Core i3 or higher
RAM	4 GB minimum (8 GB recommended)
Storage	10 GB free disk space
Other Tools	draw.io for diagrams, GitHub for version control

CHAPTER 3: DATABASE DESIGN

(Deliverables of Review – I)

3.1 Identified Entities

The Hospital Management System is modeled with eight core entities. Together, they support patient registration, consultations, prescriptions, billing, ward allocation, admissions, and staffing.

Table 3.1: Entities Identified for the Selected Domain

S.No	Entity Name	Description
1	Patient	Stores patient personal and medical details such as name, age, gender, contact and medical history.
2	Doctor	Stores doctor details such as name, specialization, department, contact and availability.
3	Appointment	Links patients and doctors with the date, time and status of a consultation.
4	Prescription	Records medicines prescribed to a patient during a visit, linked to the doctor and appointment.
5	Billing	Stores billing details for treatments and services availed, linked to the patient.
6	Ward	Stores ward/room details such as ward number, type and capacity for patient admission.
7	Staff	Stores non-doctor hospital staff details, such as nurses and admin, with role and department.
8	Admission	Tracks patient admission and discharge details, linked to the patient and ward.

3.2 Entity Attributes and Attribute Types

The following table identifies the attributes used in the implemented database. The classification records the conceptual attribute type; the physical database stores each column as an atomic value.

Table 3.2: Attributes of Each Entity

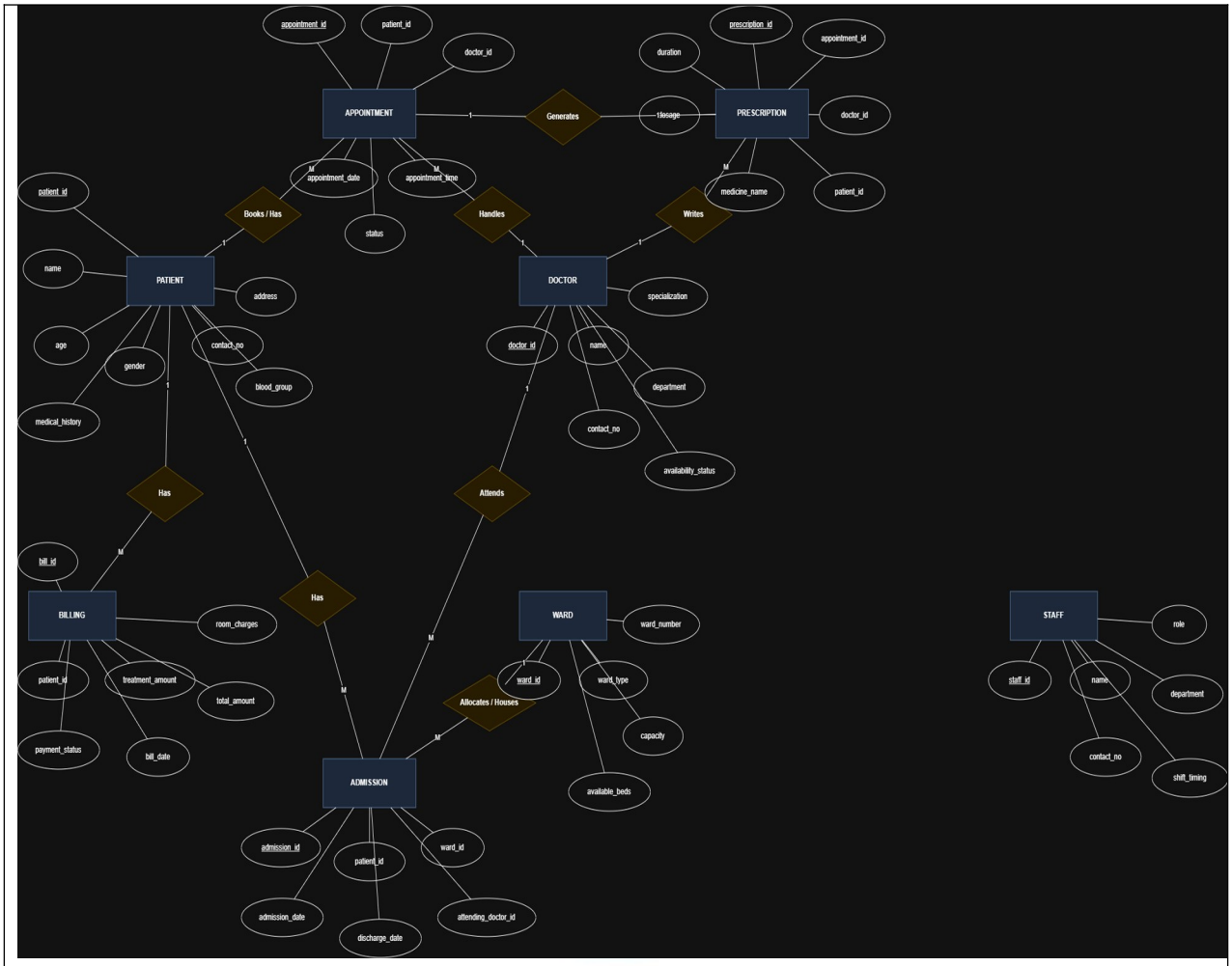
Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Patient	patient_id	INT	PK	Simple
Patient	name	VARCHAR(100)	—	Simple
Patient	age	INT	—	Simple

Patient	gender	VARCHAR(10)	—	Simple
Patient	contact_no	VARCHAR(15)	—	Simple
Patient	address	VARCHAR(200)	—	Composite
Patient	blood_group	VARCHAR(5)	—	Simple
Patient	medical_history	TEXT	—	Multi-valued
Doctor	doctor_id	INT	PK	Simple
Doctor	name	VARCHAR(100)	—	Simple
Doctor	specialization	VARCHAR(100)	—	Simple
Doctor	department	VARCHAR(100)	—	Simple
Doctor	contact_no	VARCHAR(15)	—	Simple
Doctor	availability_status	VARCHAR(20)	—	Simple
Appointment	appointment_id	INT	PK	Simple
Appointment	patient_id	INT	FK	Simple
Appointment	doctor_id	INT	FK	Simple
Appointment	appointment_date	DATE	—	Simple
Appointment	appointment_time	TIME	—	Simple
Appointment	status	VARCHAR(20)	—	Simple
Prescription	prescription_id	INT	PK	Simple
Prescription	appointment_id	INT	FK	Simple
Prescription	doctor_id	INT	FK	Simple
Prescription	patient_id	INT	FK	Simple
Prescription	medicine_name	VARCHAR(100)	—	Multi-valued
Prescription	dosage	VARCHAR(50)	—	Simple
Prescription	duration	VARCHAR(50)	—	Simple

Billing	bill_id	INT	PK	Simple
Billing	patient_id	INT	FK	Simple
Billing	treatment_amount	DECIMAL(10,2)	—	Simple
Billing	room_charges	DECIMAL(10,2)	—	Simple
Billing	total_amount	DECIMAL(10,2)	—	Derived
Billing	payment_status	VARCHAR(20)	—	Simple
Billing	bill_date	DATE	—	Simple
Ward	ward_id	INT	PK	Simple
Ward	ward_type	VARCHAR(50)	—	Simple
Ward	ward_number	VARCHAR(20)	—	Simple
Ward	capacity	INT	—	Simple
Ward	available_beds	INT	—	Derived
Staff	staff_id	INT	PK	Simple
Staff	name	VARCHAR(100)	—	Simple
Staff	role	VARCHAR(50)	—	Simple
Staff	department	VARCHAR(100)	—	Simple
Staff	contact_no	VARCHAR(15)	—	Simple
Staff	shift_timing	VARCHAR(50)	—	Simple
Admission	admission_id	INT	PK	Simple
Admission	patient_id	INT	FK	Simple
Admission	ward_id	INT	FK	Simple
Admission	admission_date	DATE	—	Simple
Admission	discharge_date	DATE	—	Simple
Admission	attending_doctor_id	INT	FK	Simple

3.3 ER Diagram

The ER diagram stored in the repository at DIAGRAMS/ER diagram.png represents the eight entities and their associations. It should be inserted here before final submission if a printed diagram is required.



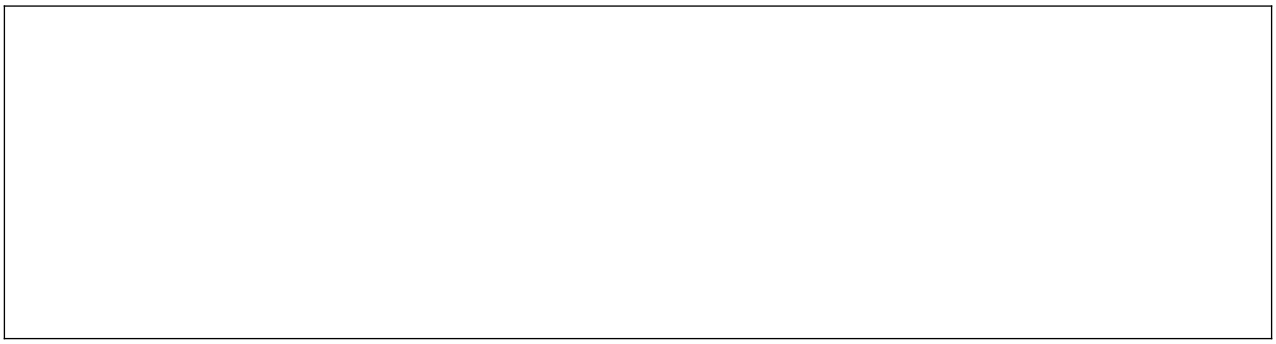


Fig. 3.1: ER Diagram of the Hospital Management System

3.4 Relationships and Cardinality

Table 3.3: Relationships and Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)	Participation
Appointment	Generates	Prescription	1:1	Partial (Appointment side)
Doctor	Handles	Appointment	1:M	Total (Appointment), Partial (Doctor)
Patient	Books / Has	Appointment	1:M	Total (Appointment), Partial (Patient)
Patient	Has	Billing	1:M	Total (Billing), Partial (Patient)
Patient	Has	Admission	1:M	Total (Admission), Partial

				(Patient)
Ward	Allocates / Houses	Admission	1:M	Total (Admission), Partial (Ward)
Doctor	Attends	Admission	1:M	Total (Admission), Partial (Doctor)
Doctor	Writes	Prescription	1:M	Total (Prescription), Partial (Doctor)

3.5 Relational Database Schema

The relational schema below is derived directly from SQL/DDL.sql in the project repository. It lists the columns and constraints implemented in the current MySQL script.

Table 3.4: Relational Database Schema

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Patient	patient_id	INT	PK	NOT NULL, AUTO_INCREMENT
Patient	name	VARCHAR(100)	—	NOT NULL
Patient	age	INT	—	CHECK (age > 0)
Patient	gender	VARCHAR(10)	—	NOT NULL
Patient	contact_no	VARCHAR(15)	—	UNIQUE,

				NOT NULL
Patient	address	VARCHAR(200)	—	—
Patient	blood_group	VARCHAR(5)	—	—
Patient	medical_history	TEXT	—	—
Doctor	doctor_id	INT	PK	NOT NULL, AUTO_INCREMENT
Doctor	name	VARCHAR(100)	—	NOT NULL
Doctor	specialization	VARCHAR(100)	—	NOT NULL
Doctor	department	VARCHAR(100)	—	NOT NULL
Doctor	contact_no	VARCHAR(15)	—	UNIQUE, NOT NULL
Doctor	availability_status	VARCHAR(20)	—	DEFAULT 'Available'
Appointment	appointment_id	INT	PK	NOT NULL, AUTO_INCREMENT
Appointment	patient_id	INT	FK	REFERENCES Patient(patient_id)
Appointment	doctor_id	INT	FK	REFERENCES Doctor(doctor_id)
Appointment	appointment_date	DATE	—	NOT NULL
Appointment	appointment_time	TIME	—	NOT NULL
Appointment	status	VARCHAR(20)	—	DEFAULT

				'Scheduled'
Prescription	prescription_id	INT	PK	NOT NULL, AUTO_INCREMENT
Prescription	appointment_id	INT	FK	REFERENCES Appointment(appointment_id)
Prescription	doctor_id	INT	FK	REFERENCES Doctor(doctor_id)
Prescription	patient_id	INT	FK	REFERENCES Patient(patient_id)
Prescription	medicine_name	VARCHAR(100)	—	NOT NULL
Prescription	dosage	VARCHAR(50)	—	—
Prescription	duration	VARCHAR(50)	—	—
Billing	bill_id	INT	PK	NOT NULL, AUTO_INCREMENT
Billing	patient_id	INT	FK	REFERENCES Patient(patient_id)
Billing	treatment_amount	DECIMAL(10,2)	—	CHECK (treatment_amount >= 0)
Billing	room_charges	DECIMAL(10,2)	—	CHECK (room_charges >= 0)
Billing	total_amount	DECIMAL(10,2	—	NOT NULL

		)		
Billing	payment_status	VARCHAR(20)	—	DEFAULT 'Pending'
Billing	bill_date	DATE	—	NOT NULL
Ward	ward_id	INT	PK	NOT NULL, AUTO_INCREMENT
Ward	ward_type	VARCHAR(50)	—	NOT NULL
Ward	ward_number	VARCHAR(20)	—	UNIQUE
Ward	capacity	INT	—	CHECK (capacity > 0)
Ward	available_beds	INT	—	CHECK (available_beds >= 0)
Staff	staff_id	INT	PK	NOT NULL, AUTO_INCREMENT
Staff	name	VARCHAR(100)	—	NOT NULL
Staff	role	VARCHAR(50)	—	NOT NULL
Staff	department	VARCHAR(100)	—	NOT NULL
Staff	contact_no	VARCHAR(15)	—	UNIQUE
Staff	shift_timing	VARCHAR(50)	—	—
Admission	admission_id	INT	PK	NOT NULL, AUTO_INCREMENT
Admission	patient_id	INT	FK	REFERENCES Patient(patient

				_id)
Admission	ward_id	INT	FK	REFERENCES Ward(ward_id)
Admission	admission_date	DATE	—	NOT NULL
Admission	discharge_date	DATE	—	—
Admission	attending_doctor_id	INT	FK	REFERENCES Doctor(doctor_id)

Schema Diagram

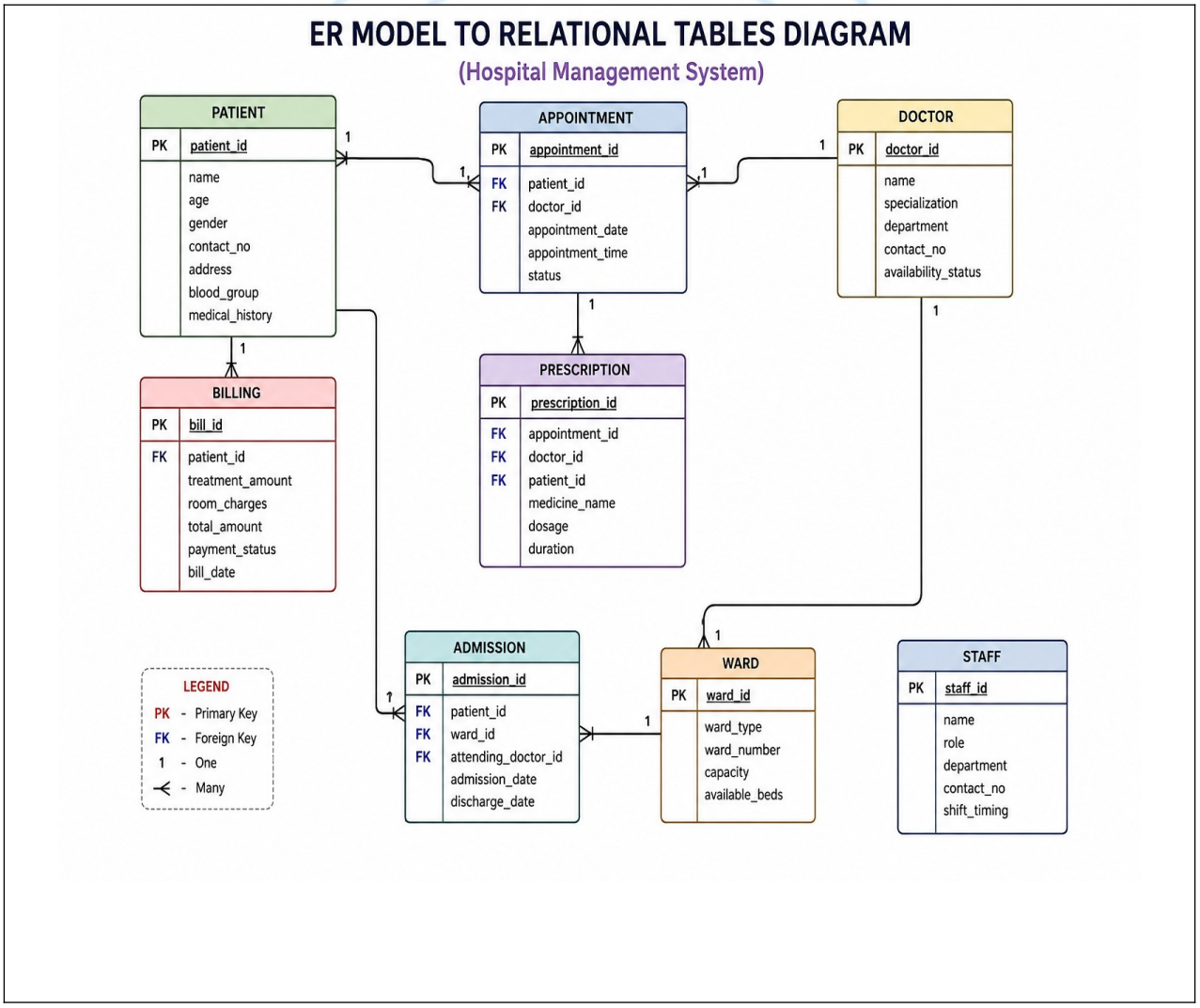


Fig. 3.2: Relational Schema Diagram

3.6 Primary Keys and Foreign Keys

Table 3.5: Primary Key and Foreign Key Definitions

Table	Primary Key	Foreign Key	References (Table.Column)
Patient	patient_id	—	—
Doctor	doctor_id	—	—
Appointment	appointment_id	patient_id, doctor_id	Patient.patient_id, Doctor.doctor_id
Prescription	prescription_id	appointment_id, doctor_id, patient_id	Appointment.appointment_id, Doctor.doctor_id, Patient.patient_id
Billing	bill_id	patient_id	Patient.patient_id
Ward	ward_id	—	—
Admission	admission_id	patient_id, ward_id, attending_doctor_id	Patient.patient_id, Ward.ward_id, Doctor.doctor_id
Staff	staff_id	—	—

3.7 Normalization up to 3NF

Consider an unnormalized single-relation view of the hospital data, where one row combined a patient's details together with all of their appointments, prescriptions and billing information. This unnormalized relation (UNF) has repeating groups — a patient with multiple appointments would repeat the appointment_date, doctor_id and status values within the same row — which was resolved by moving to 1NF, where every attribute holds a single atomic value and each appointment is stored as its own row. At this stage, however, non-key attributes such as a doctor's specialization still depended only on doctor_id and not on the whole key, causing update anomalies (changing a doctor's specialization would require updating it in every appointment row). Moving to 2NF

removed this partial dependency by splitting Doctor into its own table, so that doctor-specific attributes depend only on doctor_id. Finally, transitive dependencies were checked — for example, in Billing, total_amount is functionally dependent on treatment_amount and room_charges rather than directly on bill_id — and the design was organized in 3NF so that every non-key attribute depends only on the primary key of its own table and not on another non-key attribute. The resulting eight tables (Patient, Doctor, Appointment, Prescription, Billing, Ward, Staff, Admission) are each in 3NF, eliminating insertion, update and deletion anomalies present in the original unnormalized structure.

UNF → 1NF → 2NF → 3NF

Table 3.6: Normalization Steps

Normal Form	Condition Satisfied	Anomaly Removed	Resulting Tables
UNF	Contains repeating groups (a patient row repeats appointment/prescription details)	Repetition of appointment and prescription data within a single patient row	Single unnormalized hospital relation
1NF	All attributes hold atomic, single values; no repeating groups	Removes repeating groups by giving each appointment its own row	Patient, Appointment (combined with other details)
2NF	In 1NF, and every non-key attribute depends on the whole primary key	Removes partial dependency (e.g., doctor details no longer repeated per appointment)	Patient, Doctor, Appointment (separated)
3NF	In 2NF, and no transitive dependency of non-key attributes on other non-key attributes	Removes transitive dependency (e.g., total_amount depends on treatment_amount/room_charges, not bill_id	Patient, Doctor, Appointment, Prescription, Billing, Ward, Staff, Admission

		directly)	

CHAPTER 4: DATABASE IMPLEMENTATION

(Deliverables of Review – II)

4.1 Database and Table Creation – DDL

The full DDL implementation is maintained in the repository at SQL/DDL.sql. It creates the hospital_management database and the eight tables shown in Chapter 3.

```
-- Create database
DROP DATABASE IF EXISTS hospital_management;
CREATE DATABASE hospital_management;
USE hospital_management;
-- ============================================================
-- 1. PATIENT TABLE
-- ============================================================
CREATE TABLE PATIENT (
patient_id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
age INT,
gender VARCHAR(10),
contact_no VARCHAR(15),
address VARCHAR(100),
blood_group VARCHAR(5),
medical_history VARCHAR(200)
);
-- ============================================================
-- 2. DOCTOR TABLE
-- ============================================================
CREATE TABLE DOCTOR (
doctor_id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
specialization VARCHAR(50),
```

```
department VARCHAR(50),
contact_no VARCHAR(15),
availability_status VARCHAR(20)
);
-- ============================================================
-- 3. WARD TABLE
-- ============================================================
CREATE TABLE WARD (
ward_id INT PRIMARY KEY,
ward_type VARCHAR(30),
ward_number VARCHAR(10),
capacity INT,
available_beds INT
);
-- ============================================================
-- 4. STAFF TABLE
-- ============================================================
CREATE TABLE STAFF (
staff_id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
role VARCHAR(50),
department VARCHAR(50),
contact_no VARCHAR(15),
shift_timing VARCHAR(30)
);
-- ============================================================
-- 5. APPOINTMENT TABLE
-- ============================================================
CREATE TABLE APPOINTMENT (
appointment_id INT PRIMARY KEY,
patient_id INT,
```

```
    doctor_id INT,
    appointment_date DATE,
    appointment_time TIME,
    status VARCHAR(20),
    FOREIGN KEY (patient_id)
    REFERENCES PATIENT(patient_id),
    FOREIGN KEY (doctor_id)
    REFERENCES DOCTOR(doctor_id)
);
-- ============================================================
-- 6. BILLING TABLE
-- ============================================================
CREATE TABLE BILLING (
    bill_id INT PRIMARY KEY,
    patient_id INT,
    treatment_amount DECIMAL(10,2),
    room_charges DECIMAL(10,2),
    total_amount DECIMAL(10,2),
    payment_status VARCHAR(20),
    bill_date DATE,
    FOREIGN KEY (patient_id)
    REFERENCES PATIENT(patient_id)
);
-- ============================================================
-- 7. ADMISSION TABLE
-- ============================================================
CREATE TABLE ADMISSION (
    admission_id INT PRIMARY KEY,
    patient_id INT,
    ward_id INT,
    attending_doctor_id INT,
```

```
    admission_date DATE,
    discharge_date DATE,
    FOREIGN KEY (patient_id)
    REFERENCES PATIENT(patient_id),
    FOREIGN KEY (ward_id)
    REFERENCES WARD(ward_id),
    FOREIGN KEY (attending_doctor_id)
    REFERENCES DOCTOR(doctor_id)
);
-- ============================================================
-- 8. PRESCRIPTION TABLE
-- ============================================================
CREATE TABLE PRESCRIPTION (
    prescription_id INT PRIMARY KEY,
    appointment_id INT,
    doctor_id INT,
    patient_id INT,
    medicine_name VARCHAR(50),
    dosage VARCHAR(30),
    duration VARCHAR(30),
    FOREIGN KEY (appointment_id)
    REFERENCES APPOINTMENT(appointment_id),
    FOREIGN KEY (doctor_id)
    REFERENCES DOCTOR(doctor_id),
    FOREIGN KEY (patient_id)
    REFERENCES PATIENT(patient_id)
);
```

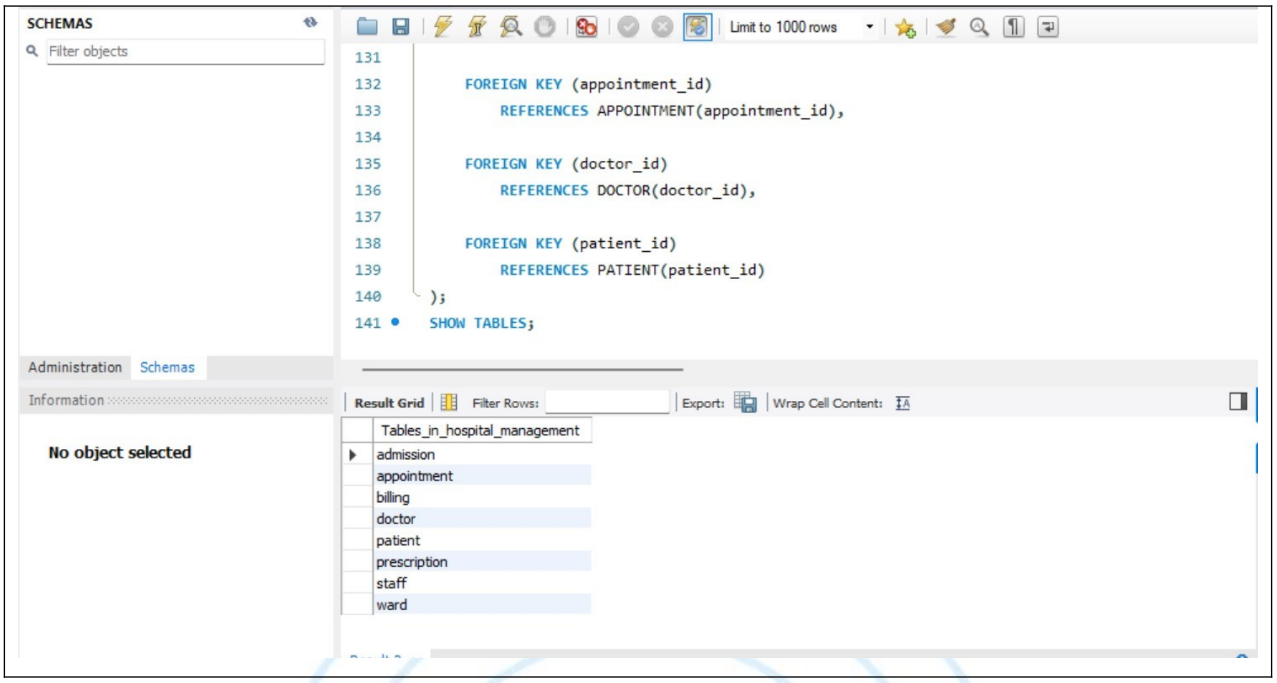


Fig. 4.1: Successful Creation of Tables

4.2 Sample Data Insertion – DML

SQL/DML.sql inserts ten sample records into each of the eight tables. The data set is used to demonstrate retrieval, filtering, sorting, aggregate operations, and relationships between tables.

Table 4.1: Sample Data Summary

S.No	Table Name	No. of Records Inserted	Screenshot Reference
1	Patient	10	Fig.4.2.1
2	Doctor	10	Fig.4.2.2
3	Ward	10	Fig.4.2.3
4	Staff	10	Fig.4.2.4
5	Appointment	10	Fig.4.2.5
6	Billing	10	Fig.4.2.6
7	Admission	10	Fig.4.2.7
8	Prescription	10	Fig.4.2.8

INSER

T INTO PATIENT

(patient_id, name, age, gender, contact_no, address, blood_group, medical_history)

VALUES

(1, 'Ravi Kumar', 25, 'Male', '9876543210', 'Hyderabad', 'B+', 'Fever'),

(2, 'Anjali Rao', 32, 'Female', '9876543211', 'Vijayawada', 'A+', 'Diabetes'),

(3, 'Suresh Reddy', 45, 'Male', '9876543212', 'Guntur', 'O+', 'Hypertension'),

(4, 'Priya Sharma', 28, 'Female', '9876543213', 'Visakhapatnam', 'AB+', 'Asthma'),

(5, 'Arjun Singh', 36, 'Male', '9876543214', 'Warangal', 'B-', 'None'),

(6, 'Lakshmi Devi', 52, 'Female', '9876543215', 'Kakinada', 'A-', 'Diabetes'),

(7, 'Kiran Kumar', 41, 'Male', '9876543216', 'Rajahmundry', 'O-', 'Heart Disease'),

(8, 'Sneha Patel', 23, 'Female', '9876543217', 'Nellore', 'B+', 'Migraine'),

(9, 'Manoj Rao', 60, 'Male', '9876543218', 'Tirupati', 'AB-', 'Arthritis'),

(10, 'Divya Reddy', 30, 'Female', '9876543219', 'Amalapuram', 'O+', 'None')

patient_id	name	age	gender	contact_no	address	blood_group	medical_history
1	Ravi Kumar	25	Male	9876543210	Hyderabad	B+	Fever
2	Anjali Rao	32	Female	9876543211	Vijayawada	A+	Diabetes
3	Suresh Reddy	45	Male	9876543212	Guntur	O+	Hypertension
4	Priya Sharma	28	Female	9876543213	Visakhapatnam	AB+	Asthma
5	Arjun Singh	36	Male	9876543214	Warangal	B-	None
6	Lakshmi Devi	52	Female	9876543215	Kakinada	A-	Diabetes
7	Kiran Kumar	41	Male	9876543216	Rajahmundry	O-	Heart Disease
8	Sneha Patel	23	Female	9876543217	Nellore	B+	Migraine
9	Manoj Rao	60	Male	9876543218	Tirupati	AB-	Arthritis
10	Divya Reddy	30	Female	9876543219	Amalapuram	O+	None
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig.4.2.1

INSERT INTO DOCTOR

(doctor_id, name, specialization, department, contact_no, availability_status)

VALUES

(101, 'Dr. Ramesh', 'Cardiologist', 'Cardiology', '9000000001', 'Available'),

(102, 'Dr. Suresh', 'Neurologist', 'Neurology', '9000000002', 'Available'),

(103, 'Dr. Priya', 'Pediatrician', 'Pediatrics', '9000000003', 'Unavailable'),

(104, 'Dr. Anil', 'Orthopedic', 'Orthopedics', '9000000004', 'Available'),

(105, 'Dr. Kavya', 'Dermatologist', 'Dermatology', '9000000005', 'Available'),

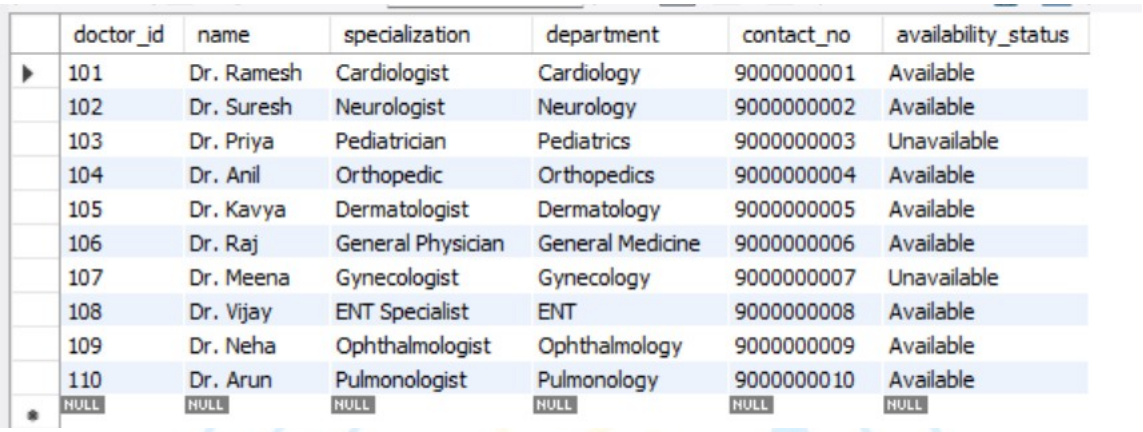
(106, 'Dr. Raj', 'General Physician', 'General Medicine', '9000000006', 'Available'),

(107, 'Dr. Meena', 'Gynecologist', 'Gynecology', '9000000007', 'Unavailable'),

(108, 'Dr. Vijay', 'ENT Specialist', 'ENT', '9000000008', 'Available'),

(109, 'Dr. Neha', 'Ophthalmologist', 'Ophthalmology', '9000000009', 'Available'),

(110, 'Dr. Arun', 'Pulmonologist', 'Pulmonology', '9000000010', 'Available');



doctor_id	name	specialization	department	contact_no	availability_status
101	Dr. Ramesh	Cardiologist	Cardiology	9000000001	Available
102	Dr. Suresh	Neurologist	Neurology	9000000002	Available
103	Dr. Priya	Pediatrician	Pediatrics	9000000003	Unavailable
104	Dr. Anil	Orthopedic	Orthopedics	9000000004	Available
105	Dr. Kavya	Dermatologist	Dermatology	9000000005	Available
106	Dr. Raj	General Physician	General Medicine	9000000006	Available
107	Dr. Meena	Gynecologist	Gynecology	9000000007	Unavailable
108	Dr. Vijay	ENT Specialist	ENT	9000000008	Available
109	Dr. Neha	Ophthalmologist	Ophthalmology	9000000009	Available
110	Dr. Arun	Pulmonologist	Pulmonology	9000000010	Available
NULL	NULL	NULL	NULL	NULL	NULL

Fig.4.2.2

INSERT INTO WARD

(ward_id, ward_type, ward_number, capacity, available_beds)

VALUES

(201, 'General', 'G01', 20, 12),

(202, 'General', 'G02', 20, 15),

(203, 'ICU', 'I01', 10, 4),

(204, 'ICU', 'I02', 10, 6),

(205, 'Private', 'P01', 5, 2),

(206, 'Private', 'P02', 5, 3),

(207, 'Emergency', 'E01', 15, 8),

(208, 'Maternity', 'M01', 12, 7),

(209, 'Pediatric', 'PD01', 10, 5),

(210, 'General', 'G03', 20, 14);

ward_id	ward_type	ward_number	capacity	available_beds
201	General	G01	20	12
202	General	G02	20	15
203	ICU	I01	10	4
204	ICU	I02	10	6
205	Private	P01	5	2
206	Private	P02	5	3
207	Emergency	E01	15	8
208	Maternity	M01	12	7
209	Pediatric	PD01	10	5
210	General	G03	20	14
NULL	NULL	NULL	NULL	NULL

Fig.4.2.3

INSERT INTO STAFF

(staff_id, name, role, department, contact_no, shift_timing)

VALUES

(601, 'Ramesh Kumar', 'Nurse', 'General Medicine', '9100000001', 'Morning'),

(602, 'Sita Devi', 'Nurse', 'Cardiology', '9100000002', 'Evening'),

(603, 'Mohan Rao', 'Receptionist', 'Administration', '9100000003', 'Morning'),

(604, 'Geetha Sharma', 'Nurse', 'ICU', '9100000004', 'Night'),

(605, 'Rajesh Kumar', 'Lab Technician', 'Laboratory', '9100000005', 'Morning'),

(606, 'Anita Reddy', 'Pharmacist', 'Pharmacy', '9100000006', 'Evening'),

(607, 'Karthik Singh', 'Nurse', 'Emergency', '9100000007', 'Night'),

(608, 'Meena Rao', 'Receptionist', 'Administration', '9100000008', 'Evening'),

(609, 'Vijay Kumar', 'Lab Technician', 'Laboratory', '9100000009', 'Morning'),

(610, 'Pooja Devi', 'Nurse', 'Pediatrics', '9100000010', 'Morning');

staff_id	name	role	department	contact_no	shift_timing
601	Ramesh Kumar	Nurse	General Medicine	9100000001	Morning
602	Sita Devi	Nurse	Cardiology	9100000002	Evening
603	Mohan Rao	Receptionist	Administration	9100000003	Morning
604	Geetha Sharma	Nurse	ICU	9100000004	Night
605	Rajesh Kumar	Lab Technician	Laboratory	9100000005	Morning
606	Anita Reddy	Pharmacist	Pharmacy	9100000006	Evening
607	Karthik Singh	Nurse	Emergency	9100000007	Night
608	Meena Rao	Receptionist	Administration	9100000008	Evening
609	Vijay Kumar	Lab Technician	Laboratory	9100000009	Morning
610	Pooja Devi	Nurse	Pediatrics	9100000010	Morning
NULL	NULL	NULL	NULL	NULL	NULL

Fig.4.2.4

INSERT INTO APPOINTMENT

(appointment_id, patient_id, doctor_id, appointment_date, appointment_time, status)

VALUES

(1001, 1, 106, '2026-09-01', '09:00:00', 'Completed'),

(1002, 2, 101, '2026-09-02', '10:00:00', 'Completed'),

(1003, 3, 102, '2026-09-03', '11:00:00', 'Scheduled'),

(1004, 4, 105, '2026-09-04', '09:30:00', 'Completed'),

(1005, 5, 104, '2026-09-05', '12:00:00', 'Scheduled'),

(1006, 6, 106, '2026-09-06', '10:30:00', 'Completed'),

(1007, 7, 101, '2026-09-07', '14:00:00', 'Scheduled'),

(1008, 8, 102, '2026-09-08', '15:00:00', 'Completed'),

(1009, 9, 104, '2026-09-09', '11:30:00', 'Cancelled'),

(1010, 10, 110, '2026-09-10', '16:00:00', 'Scheduled');

appointment_id	patient_id	doctor_id	appointment_date	appointment_time	status
1001	1	106	2026-09-01	09:00:00	Completed
1002	2	101	2026-09-02	10:00:00	Completed
1003	3	102	2026-09-03	11:00:00	Scheduled
1004	4	105	2026-09-04	09:30:00	Completed
1005	5	104	2026-09-05	12:00:00	Scheduled
1006	6	106	2026-09-06	10:30:00	Completed
1007	7	101	2026-09-07	14:00:00	Scheduled
1008	8	102	2026-09-08	15:00:00	Completed
1009	9	104	2026-09-09	11:30:00	Cancelled
1010	10	110	2026-09-10	16:00:00	Scheduled
NULL	NULL	NULL	NULL	NULL	NULL

Fig.4.2.5

INSERT INTO BILLING

(bill_id, patient_id, treatment_amount, room_charges, total_amount, payment_status, bill_date)

VALUES

(5001, 1, 2000.00, 1000.00, 3000.00, 'Paid', '2026-09-01'),

(5002, 2, 3500.00, 1500.00, 5000.00, 'Paid', '2026-09-02'),

(5003, 3, 5000.00, 2000.00, 7000.00, 'Pending', '2026-09-03'),

(5004, 4, 2500.00, 1000.00, 3500.00, 'Paid', '2026-09-04'),

(5005, 5, 4000.00, 1500.00, 5500.00, 'Pending', '2026-09-05'),

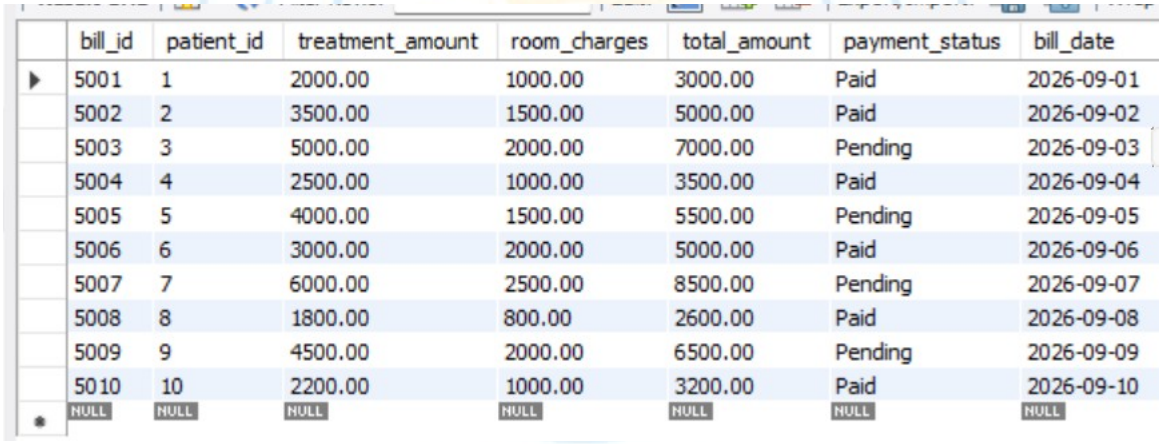
(5006, 6, 3000.00, 2000.00, 5000.00, 'Paid', '2026-09-06'),

(5007, 7, 6000.00, 2500.00, 8500.00, 'Pending', '2026-09-07'),

(5008, 8, 1800.00, 800.00, 2600.00, 'Paid', '2026-09-08'),

(5009, 9, 4500.00, 2000.00, 6500.00, 'Pending', '2026-09-09'),

(5010, 10, 2200.00, 1000.00, 3200.00, 'Paid', '2026-09-10');



bill_id	patient_id	treatment_amount	room_charges	total_amount	payment_status	bill_date
5001	1	2000.00	1000.00	3000.00	Paid	2026-09-01
5002	2	3500.00	1500.00	5000.00	Paid	2026-09-02
5003	3	5000.00	2000.00	7000.00	Pending	2026-09-03
5004	4	2500.00	1000.00	3500.00	Paid	2026-09-04
5005	5	4000.00	1500.00	5500.00	Pending	2026-09-05
5006	6	3000.00	2000.00	5000.00	Paid	2026-09-06
5007	7	6000.00	2500.00	8500.00	Pending	2026-09-07
5008	8	1800.00	800.00	2600.00	Paid	2026-09-08
5009	9	4500.00	2000.00	6500.00	Pending	2026-09-09
5010	10	2200.00	1000.00	3200.00	Paid	2026-09-10
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig.4.2.6

INSERT INTO ADMISSION

(admission_id, patient_id, ward_id, attending_doctor_id, admission_date, discharge_date)

VALUES

(3001, 1, 201, 106, '2026-09-01', '2026-09-03'),

(3002, 2, 202, 101, '2026-09-02', '2026-09-05'),

(3003, 3, 203, 101, '2026-09-03', NULL),

(3004, 4, 205, 105, '2026-09-04', '2026-09-06'),

(3005, 5, 206, 104, '2026-09-05', '2026-09-08'),

(3006, 6, 201, 106, '2026-09-06', '2026-09-10'),

(3007, 7, 204, 101, '2026-09-07', NULL),

(3008, 8, 209, 102, '2026-09-08', '2026-09-09'),

(3009, 9, 202, 104, '2026-09-09', NULL),

(3010, 10, 207, 110, '2026-09-10', '2026-09-12');

admission_id	patient_id	ward_id	attending_doctor_id	admission_date	discharge_date
3001	1	201	106	2026-09-01	2026-09-03
3002	2	202	101	2026-09-02	2026-09-05
3003	3	203	101	2026-09-03	NULL
3004	4	205	105	2026-09-04	2026-09-06
3005	5	206	104	2026-09-05	2026-09-08
3006	6	201	106	2026-09-06	2026-09-10
3007	7	204	101	2026-09-07	NULL
3008	8	209	102	2026-09-08	2026-09-09
3009	9	202	104	2026-09-09	NULL
3010	10	207	110	2026-09-10	2026-09-12
NULL	NULL	NULL	NULL	NULL	NULL

Fig.4.2.7

INSERT INTO PRESCRIPTION

(prescription_id, appointment_id, doctor_id, patient_id, medicine_name, dosage, duration)

VALUES

(4001, 1001, 106, 1, 'Paracetamol', '500 mg', '5 days'),

(4002, 1002, 101, 2, 'Metformin', '500 mg', '30 days'),

(4003, 1003, 102, 3, 'Aspirin', '75 mg', '15 days'),

(4004, 1004, 105, 4, 'Cetirizine', '10 mg', '7 days'),

(4005, 1005, 104, 5, 'Ibuprofen', '400 mg', '5 days'),

(4006, 1006, 106, 6, 'Metformin', '500 mg', '30 days'),

(4007, 1007, 101, 7, 'Atorvastatin', '20 mg', '30 days'),

(4008, 1008, 102, 8, 'Sumatriptan', '50 mg', '5 days'),

(4009, 1009, 104, 9, 'Calcium', '500 mg', '20 days'),

(4010, 1010, 110, 10, 'Montelukast', '10 mg', '10 days');

prescription_id	appointment_id	doctor_id	patient_id	medicine_name	dosage	duration
4001	1001	106	1	Paracetamol	500 mg	5 days
4002	1002	101	2	Metformin	500 mg	30 days
4003	1003	102	3	Aspirin	75 mg	15 days
4004	1004	105	4	Cetirizine	10 mg	7 days
4005	1005	104	5	Ibuprofen	400 mg	5 days
4006	1006	106	6	Metformin	500 mg	30 days
4007	1007	101	7	Atorvastatin	20 mg	30 days
4008	1008	102	8	Sumatriptan	50 mg	5 days
4009	1009	104	9	Calcium	500 mg	20 days
4010	1010	110	10	Montelukast	10 mg	10 days
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig.4.2.8

Fig. 4.2: Sample Data Inserted into the Tables

4.3 Constraints and Data Integrity

Table 4.2: Constraints and Data Integrity Rules

Constraint / Validation	Applied On	Purpose	Evidence
Primary key	All eight tables	Prevents duplicate IDs and uniquely identifies each row	Implemented in SQL/DDL.sql
Foreign key	Appointment.patient_id	Prevents an appointment for a non-existent patient	References PATIENT(patient_id)
Foreign key	Appointment.doctor_id	Prevents an appointment for a non-existent doctor	References DOCTOR(doctor_id)
Foreign key	Prescription appointment doctor patient IDs	Keeps each prescription linked to valid consultation records	Implemented in SQL/DDL.sql
Foreign key	Billing.patient_id	Links every bill to a valid patient	Implemented in SQL/DDL.sql
Foreign key	Admission patient ward doctor IDs	Ensures admissions refer to valid patient, ward, and doctor records	Implemented in SQL/DDL.sql
Recommended validation	All business columns	Add NOT NULL, UNIQUE, CHECK, and DEFAULT constraints before deployment	Not yet in repository DDL

4.4 Data Validation Rules

The implemented DDL uses primary keys to prevent duplicate identifiers and foreign keys to prevent orphan appointment, prescription, billing, and admission records. The current repository script does not yet declare NOT NULL, UNIQUE, CHECK, or DEFAULT constraints; these are recommended hardening improvements, not implemented constraints.

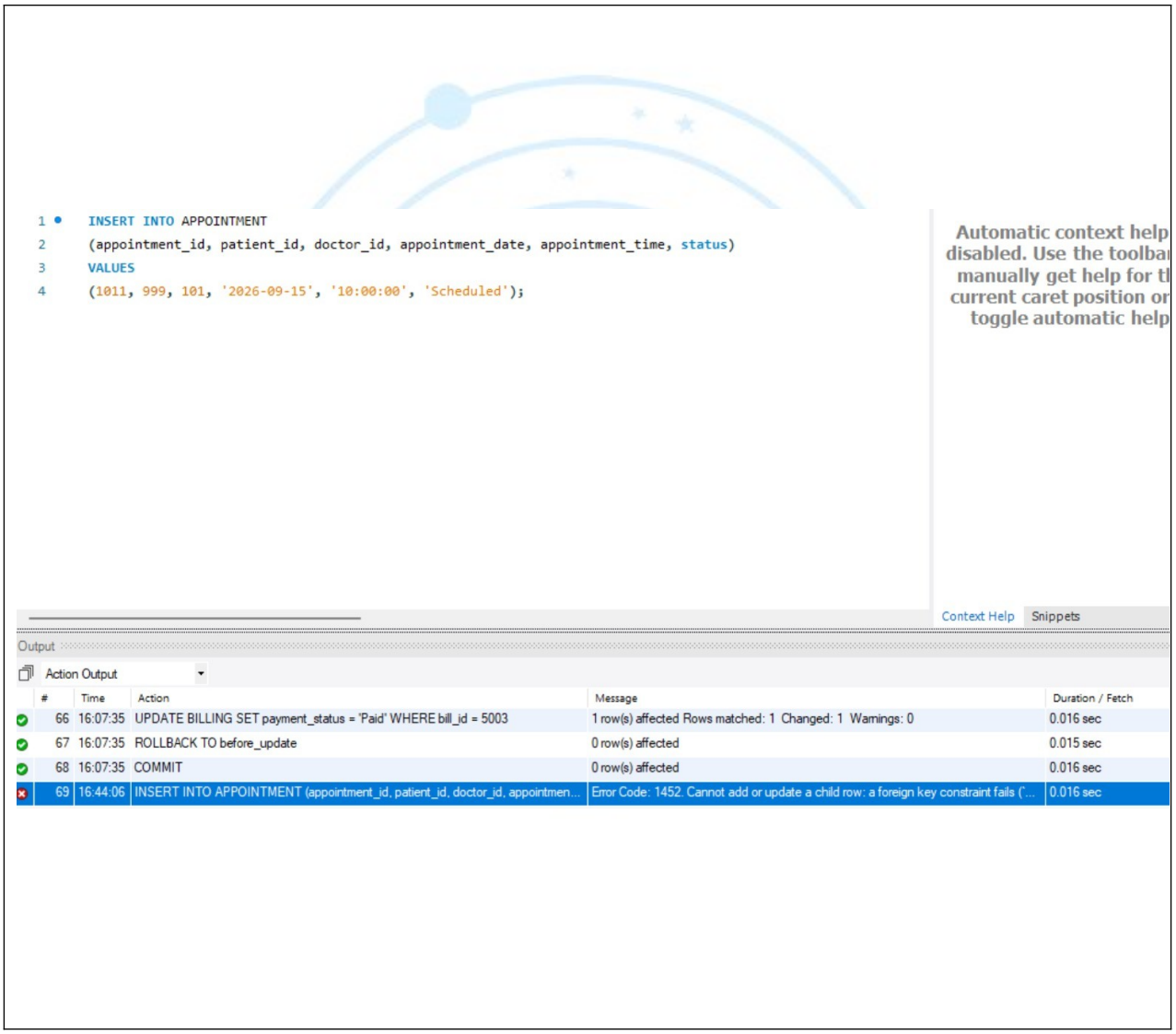


Fig. 4.3: Referential integrity enforced when an appointment references a non-existent patient.

CHAPTER 5: SQL OPERATIONS AND QUERIES

The repository currently contains basic retrieval and aggregate queries in SQL/DQL.sql. The following tables document those queries and provide the additional SQL required to satisfy the capstone coverage for joins, subqueries, views, set operations, DCL, and TCL. Execute the additional statements in MySQL Workbench and commit them before final submission.

5.1 DQL – Basic Retrieval Queries

Table 5.1: Basic Retrieval Queries

Q.No	Purpose	SQL Statement	Output Ref.
Q1	Display all patient records	SELECT * FROM PATIENT;	Fig. 7.1
Q2	Find female patients older than 30	SELECT * FROM PATIENT WHERE age > 30 AND gender = 'Female';	Fig. 7.2
Q3	Find patients younger than 30 or older than 50	SELECT * FROM PATIENT WHERE age < 30 OR age > 50;	Fig. 7.3
Q4	Display available doctors	SELECT * FROM DOCTOR WHERE availability_status = 'Available';	Fig. 7.4
Q5	Sort patients from youngest to oldest	SELECT * FROM PATIENT ORDER BY age ASC;	Fig. 7.5
Q6	List distinct blood groups	SELECT DISTINCT blood_group FROM PATIENT;	Fig. 7.6

5.2 Join Operations

The following join operations demonstrate how related hospital records are combined. MySQL does not provide a native FULL OUTER JOIN, so Q10 uses a UNION of left and right joins to achieve the equivalent result.

Table 5.2: Join Operations

Q.No	Type of Join	SQL Statement	Output Ref.
Q7	Equi / Inner join	SELECT a.appointment_id, p.name AS patient, d.name AS doctor, a.appointment_date FROM APPOINTMENT a INNER JOIN PATIENT p ON a.patient_id = p.patient_id INNER JOIN DOCTOR d ON a.doctor_id = d.doctor_id;	Fig. 7.7
Q8	Left outer join	SELECT p.patient_id, p.name, a.appointment_id, a.status FROM PATIENT p LEFT JOIN APPOINTMENT a ON	Fig. 7.8

		p.patient_id = a.patient_id;	
Q9	Right outer join	SELECT d.doctor_id, d.name, a.appointment_id, a.status FROM APPOINTMENT a RIGHT JOIN DOCTOR d ON a.doctor_id = d.doctor_id;	Fig. 7.9
Q10	Full outer join simulation	SELECT p.patient_id, p.name, b.bill_id FROM PATIENT p LEFT JOIN BILLING b ON p.patient_id = b.patient_id UNION SELECT p.patient_id, p.name, b.bill_id FROM PATIENT p RIGHT JOIN BILLING b ON p.patient_id = b.patient_id;	Fig. 7.10

5.3 Aggregate Functions, GROUP BY and HAVING

Table 5.3: Aggregate, GROUP BY and HAVING Queries

Q.No	Function Used	SQL Statement	Output Ref.
Q11	SUM	SELECT SUM(treatment_amount) AS total_treatment FROM BILLING;	Fig. 7.11
Q12	COUNT and GROUP BY	SELECT gender, COUNT(*) AS total_patients FROM PATIENT GROUP BY gender;	Fig. 7.12
Q13	AVG, GROUP BY, HAVING	SELECT payment_status, AVG(total_amount) AS average_bill FROM BILLING GROUP BY payment_status HAVING AVG(total_amount) > 4000;	Fig. 7.13

5.4 Nested and Correlated Subqueries

Table 5.4: Subqueries

Q.No	Type	SQL Statement	Output Ref.
Q14	Nested	SELECT name, age FROM PATIENT WHERE age > (SELECT AVG(age) FROM PATIENT);	Fig. 7.14
Q15	Correlated	SELECT d.name FROM DOCTOR d WHERE EXISTS (SELECT 1 FROM APPOINTMENT a WHERE a.doctor_id = d.doctor_id AND a.status = 'Completed');	Fig. 7.15

5.5 Views and Relational Set Operations

Include at least one updatable view and one non-updatable view, and the set operations UNION, INTERSECT and MINUS / EXCEPT.

Table 5.5: Views and Set Operations

Q.No	View / Set Operation	SQL Statement	Output Ref.
Q16	Updatable view	CREATE VIEW available_doctors AS SELECT doctor_id, name, specialization, department FROM DOCTOR WHERE availability_status = 'Available';	Fig. 7.16
Q17	Non-updatable view	CREATE VIEW patient_bill_summary AS SELECT p.patient_id, p.name, SUM(b.total_amount) AS total_billed FROM PATIENT p JOIN BILLING b ON p.patient_id = b.patient_id GROUP BY p.patient_id, p.name;	Fig. 7.17
Q18	UNION	SELECT patient_id FROM APPOINTMENT UNION SELECT patient_id FROM BILLING;	Fig. 7.18
Q19	INTERSECT equivalent	SELECT DISTINCT a.patient_id FROM APPOINTMENT a WHERE EXISTS (SELECT 1 FROM BILLING b WHERE b.patient_id = a.patient_id);	Fig. 7.19
Q20	EXCEPT equivalent	SELECT DISTINCT a.patient_id FROM APPOINTMENT a WHERE NOT EXISTS (SELECT 1 FROM BILLING b WHERE b.patient_id = a.patient_id);	Fig. 7.20

5.6 DCL and TCL Operations

Demonstrate GRANT and REVOKE (DCL), and COMMIT, SAVEPOINT and ROLLBACK (TCL).

Table 5.6: DCL and TCL Operations

Q.No	Command	Purpose	SQL Statement	Output Ref.
Q21	GRANT	Grant read access to a reporting user	GRANT SELECT ON hospital_management.* TO 'report_user'@'localhost';	Fig. 7.21
Q22	REVOKE	Remove reporting access	REVOKE SELECT ON hospital_management.* FROM	Fig. 7.22

			'report_user'@'localhost';	
Q23	COMMIT	Permanently save a transaction	START TRANSACTION; UPDATE APPOINTMENT SET status = 'Completed' WHERE appointment_id = 1003; COMMIT;	Fig. 7.23
Q24	SAVEPOINT and ROLLBACK	Undo a change after a savepoint	START TRANSACTION; SAVEPOINT before_update; UPDATE BILLING SET payment_status = 'Paid' WHERE bill_id = 5003; ROLLBACK TO before_update; COMMIT;	Fig. 7.24

CHAPTER 6: MODULES AND FUNCTIONALITIES

The database is organized around the following functional modules, each built on top of one or more of the eight tables described in Chapter 3.

Table 6.1: Modules and Their Functionalities

S.No	Module Name	Functionality	Tables Involved
1	Patient Management	Register new patients and maintain personal and medical detail records	Patient
2	Doctor Management	Maintain doctor profiles, specializations and availability status	Doctor
3	Appointment Scheduling	Book, update and track the status of consultations between patients and doctors	Appointment, Patient, Doctor
4	Prescription Management	Record medicines, dosage and duration prescribed during a consultation	Prescription, Appointment, Doctor, Patient
5	Billing Management	Generate and track bills for treatments, room charges and payment status	Billing, Patient
6	Ward & Admission Management	Manage ward allocation and patient admission / discharge records	Ward, Admission, Patient, Doctor
7	Staff Management	Maintain non-doctor staff records, roles and shift timings	Staff

The Patient Management module handles patient registration and keeps personal and medical details up to date. The Doctor Management module maintains doctor profiles and their real-time availability, which the Appointment Scheduling module reads from when booking a consultation between a patient and a doctor. The Prescription Management module is invoked at the end of a consultation to record the medicines, dosage and duration prescribed, linking back to the appointment, doctor and patient. The Billing Management module generates a bill for each patient

covering treatment and room charges and tracks its payment status. The Ward & Admission Management module allocates ward beds and records admission and discharge dates for patients who require in-hospital care, along with the attending doctor. Finally, the Staff Management module maintains records of nurses and administrative staff, independent of the doctor records.

CHAPTER 7: RESULTS – SAMPLE OUTPUTS AND SCREENSHOTS

Execution screenshots are available in the repository SCREENSHOTS folder. Insert the selected images below before printing the final report; use captions that identify the displayed table or query result.

patient_id	name	age	gender	contact_no	address	blood_group	medical_history
1	Ravi Kumar	25	Male	9876543210	Hyderabad	B+	Fever
2	Anjali Rao	32	Female	9876543211	Vijayawada	A+	Diabetes
3	Suresh Reddy	45	Male	9876543212	Guntur	O+	Hypertension
4	Priya Sharma	28	Female	9876543213	Visakhapatnam	AB+	Asthma
5	Arjun Singh	36	Male	9876543214	Warangal	B-	None
6	Lakshmi Devi	52	Female	9876543215	Kakinada	A-	Diabetes
7	Kiran Kumar	41	Male	9876543216	Rajahmundry	O-	Heart Disease
8	Sneha Patel	23	Female	9876543217	Nellore	B+	Migraine
9	Manoj Rao	60	Male	9876543218	Tirupati	AB-	Arthritis
10	Divya Reddy	30	Female	9876543219	Amalapuram	O+	None

Fig. 7.1: Patient records retrieved from the Patient table.

patient_id	name	age	gender	contact_no	address	blood_group	medical_history
2	Anjali Rao	32	Female	9876543211	Vijayawada	A+	Diabetes
6	Lakshmi Devi	52	Female	9876543215	Kakinada	A-	Diabetes

Fig. 7.2: Female patients above 30 years retrieved using WHERE.

patient_id	name	age	gender	contact_no	address	blood_group	medical_history
1	Ravi Kumar	25	Male	9876543210	Hyderabad	B+	Fever
4	Priya Sharma	28	Female	9876543213	Visakhapatnam	AB+	Asthma
6	Lakshmi Devi	52	Female	9876543215	Kakinada	A-	Diabetes
8	Sneha Patel	23	Female	9876543217	Nellore	B+	Migraine
9	Manoj Rao	60	Male	9876543218	Tirupati	AB-	Arthritis

Fig. 7.3: Patients younger than 30 or older than 50 retrieved using OR.

doctor_id	name	specialization	department	contact_no	availability_status
101	Dr. Ramesh	Cardiologist	Cardiology	9000000001	Available
102	Dr. Suresh	Neurologist	Neurology	9000000002	Available
104	Dr. Anil	Orthopedic	Orthopedics	9000000004	Available
105	Dr. Kavya	Dermatologist	Dermatology	9000000005	Available
106	Dr. Raj	General Physician	General Medicine	9000000006	Available
108	Dr. Vijay	ENT Specialist	ENT	9000000008	Available
109	Dr. Neha	Ophthalmologist	Ophthalmology	9000000009	Available
110	Dr. Arun	Pulmonologist	Pulmonology	9000000010	Available

Fig. 7.4: Available doctors retrieved using WHERE.

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

patient_id	name	age	gender	contact_no	address	blood_group	medical_history
8	Sneha Patel	23	Female	9876543217	Nellore	B+	Migraine
1	Ravi Kumar	25	Male	9876543210	Hyderabad	B+	Fever
4	Priya Sharma	28	Female	9876543213	Visakhapatnam	AB+	Asthma
10	Divya Reddy	30	Female	9876543219	Amalapuram	O+	None
2	Anjali Rao	32	Female	9876543211	Vijayawada	A+	Diabetes
5	Arjun Singh	36	Male	9876543214	Warangal	B-	None
7	Kiran Kumar	41	Male	9876543216	Rajahmundry	O-	Heart Disease
3	Suresh Reddy	45	Male	9876543212	Guntur	O+	Hypertension
6	Lakshmi Devi	52	Female	9876543215	Kakinada	A-	Diabetes
9	Manoj Rao	60	Male	9876543218	Tirupati	AB-	Arthritis
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Fig. 7.5: Patient records ordered from youngest to oldest.

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

blood_group
B+
A+
O+
AB+
B-
A-
O-
AB-

Fig. 7.6: Distinct blood groups retrieved from the Patient table.

Limit to 1000 rows

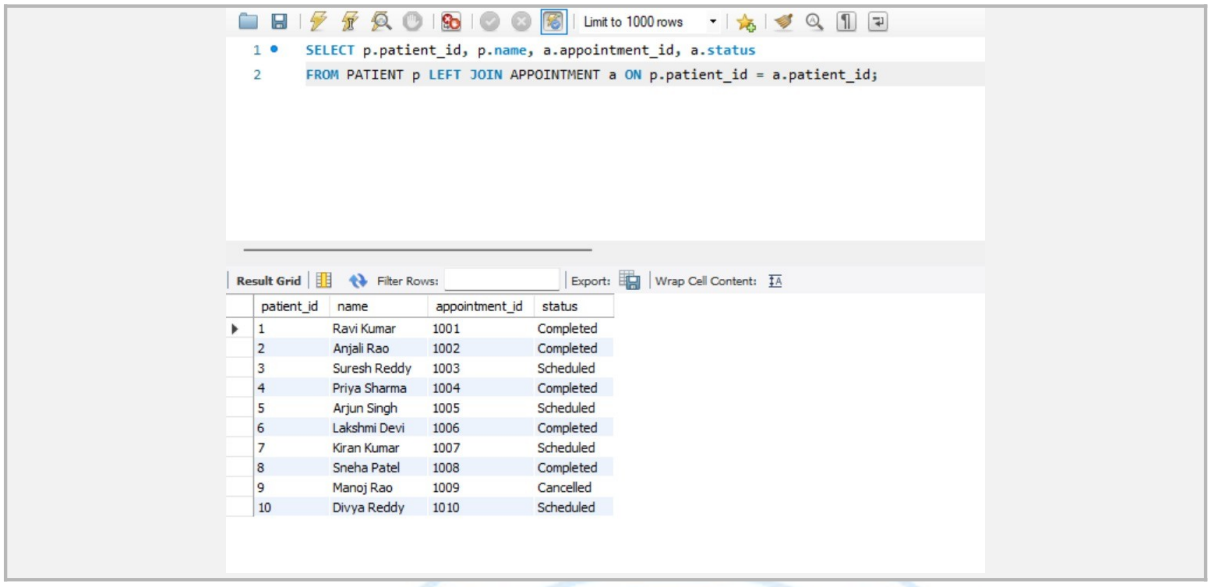
```
1  SELECT a.appointment_id, p.name AS patient, d.name AS doctor, a.appointment_date
2  FROM APPOINTMENT a INNER JOIN PATIENT p
3  ON a.patient_id = p.patient_id INNER JOIN DOCTOR d ON a.doctor_id = d.doctor_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

appointment_id	patient	doctor	appointment_date
1001	Ravi Kumar	Dr. Raj	2026-09-01
1002	Anjali Rao	Dr. Ramesh	2026-09-02
1003	Suresh Reddy	Dr. Suresh	2026-09-03
1004	Priya Sharma	Dr. Kavya	2026-09-04
1005	Arjun Singh	Dr. Anil	2026-09-05
1006	Lakshmi Devi	Dr. Raj	2026-09-06
1007	Kiran Kumar	Dr. Ramesh	2026-09-07
1008	Sneha Patel	Dr. Suresh	2026-09-08
1009	Manoj Rao	Dr. Anil	2026-09-09
1010	Divya Reddy	Dr. Arun	2026-09-10

Result 6

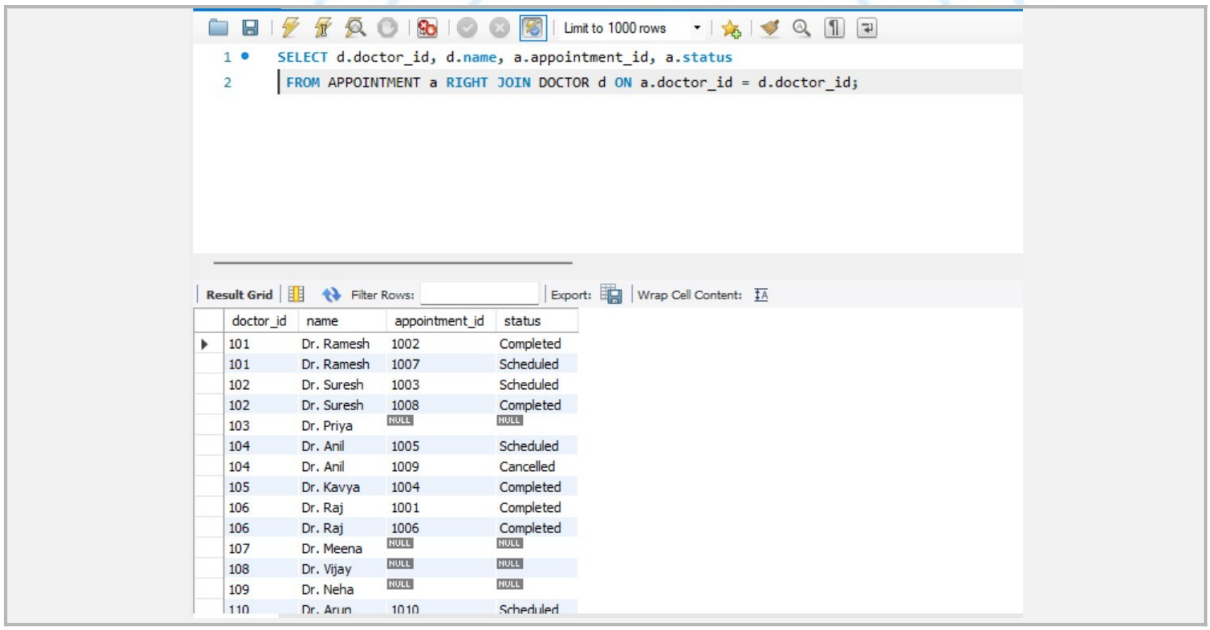
Fig. 7.7: Appointment details with patient and doctor names using INNER JOIN.



```
SELECT p.patient_id, p.name, a.appointment_id, a.status
FROM PATIENT p LEFT JOIN APPOINTMENT a ON p.patient_id = a.patient_id;
```

patient_id	name	appointment_id	status
1	Ravi Kumar	1001	Completed
2	Anjali Rao	1002	Completed
3	Suresh Reddy	1003	Scheduled
4	Priya Sharma	1004	Completed
5	Arjun Singh	1005	Scheduled
6	Lakshmi Devi	1006	Completed
7	Kiran Kumar	1007	Scheduled
8	Sneha Patel	1008	Completed
9	Manoj Rao	1009	Cancelled
10	Divya Reddy	1010	Scheduled

Fig. 7.8: Patient records with appointment information using LEFT OUTER JOIN.



```
SELECT d.doctor_id, d.name, a.appointment_id, a.status
FROM APPOINTMENT a RIGHT JOIN DOCTOR d ON a.doctor_id = d.doctor_id;
```

doctor_id	name	appointment_id	status
101	Dr. Ramesh	1002	Completed
101	Dr. Ramesh	1007	Scheduled
102	Dr. Suresh	1003	Scheduled
102	Dr. Suresh	1008	Completed
103	Dr. Priya	NULL	NULL
104	Dr. Anil	1005	Scheduled
104	Dr. Anil	1009	Cancelled
105	Dr. Kavya	1004	Completed
106	Dr. Raj	1001	Completed
106	Dr. Raj	1006	Completed
107	Dr. Meena	NULL	NULL
108	Dr. Vijay	NULL	NULL
109	Dr. Neha	NULL	NULL
110	Dr. Arun	1010	Scheduled

Fig. 7.9: Doctor records with appointment information using RIGHT OUTER JOIN.

```
1 •  SELECT p.patient_id, p.name, b.bill_id FROM PATIENT p
2    LEFT JOIN BILLING b ON p.patient_id = b.patient_id UNION
3     SELECT p.patient_id, p.name, b.bill_id FROM PATIENT p
4    RIGHT JOIN BILLING b ON p.patient_id = b.patient_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

patient_id	name	bill_id
1	Ravi Kumar	5001
2	Anjali Rao	5002
3	Suresh Reddy	5003
4	Priya Sharma	5004
5	Arjun Singh	5005
6	Lakshmi Devi	5006
7	Kiran Kumar	5007
8	Sneha Patel	5008
9	Manoj Rao	5009
10	Divya Reddy	5010

Fig. 7.10: Patient and billing records using a FULL OUTER JOIN simulation.

```
1 •  SELECT SUM(treatment_amount) AS total_treatment FROM BILLING;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_treatment
34500.00

Fig. 7.11: Total treatment amount calculated using SUM.

```
1    SELECT gender, COUNT(*) AS total_patients FROM PATIENT GROUP BY gender;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

gender	total_patients
Male	5
Female	5

Fig. 7.12: Patient count grouped by gender using GROUP BY.

```
1    SELECT payment_status, AVG(total_amount) AS average_bill FROM BILLING
2    GROUP BY payment_status HAVING AVG(total_amount) > 4000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_status	average_bill
Pending	6875.000000

Fig. 7.13: Average bill amount by payment status using HAVING.

```
1 •  SELECT name, age FROM PATIENT WHERE age > (SELECT AVG(age) FROM PATIENT)
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	age
Suresh Reddy	45
Lakshmi Devi	52
Kiran Kumar	41
Manoj Rao	60

Fig. 7.14: Patients older than the average age using a nested subquery.

```
1    SELECT d.name FROM DOCTOR d WHERE EXISTS (SELECT 1 FROM APPOINTMENT a
2     WHERE a.doctor_id = d.doctor_id AND a.status = 'Completed');
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Dr. Raj
Dr. Ramesh
Dr. Kavya
Dr. Suresh

Fig. 7.15: Doctors with completed appointments using a correlated subquery.

```
1 • USE hospital_management;
2 • SELECT doctor_id, availability_status FROM DOCTOR;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

doctor_id	availability_status
103	Unavailable
104	Available
105	Available
106	Available
107	Unavailable
108	Available
109	Available
110	Available
NULL	NULL

Fig. 7.16: Available doctors displayed through the available_doctors view.

```
1    CREATE OR REPLACE VIEW patient_bill_summary AS
2    SELECT p.patient_id, p.name, SUM(b.total_amount) AS total_billed
3    FROM PATIENT p
4    JOIN BILLING b ON p.patient_id = b.patient_id
5    GROUP BY p.patient_id, p.name;
6 •  SELECT * FROM patient_bill_summary;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

patient_id	name	total_billed
1	Ravi Kumar	3000.00
2	Anjali Rao	5000.00
3	Suresh Reddy	7000.00
4	Priya Sharma	3500.00
5	Arjun Singh	5500.00
6	Lakshmi Devi	5000.00
7	Kiran Kumar	8500.00
8	Sneha Patel	2600.00
9	Manoj Rao	6500.00
10	Divya Reddy	3200.00

Fig. 7.17: Patient billing total displayed through the patient_bill_summary view.

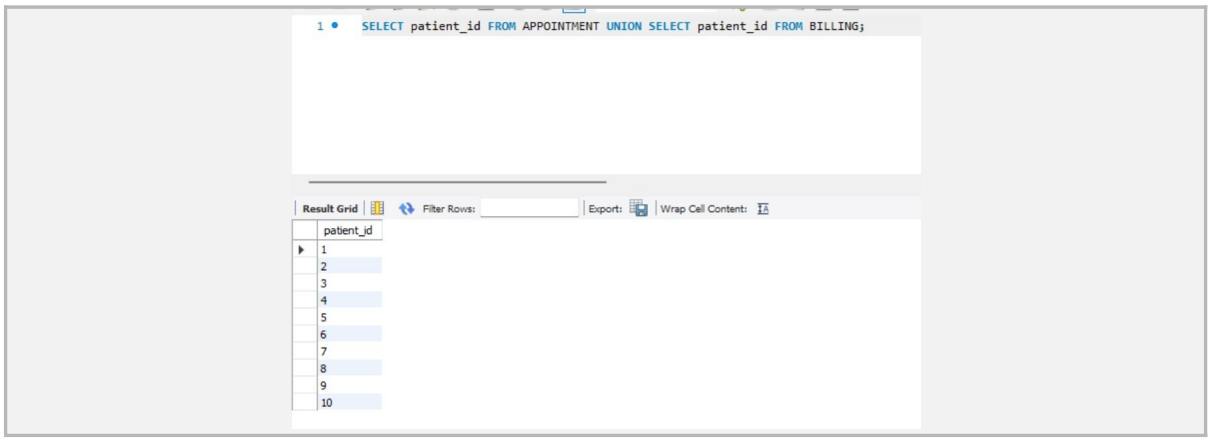


Fig. 7.18: Patient IDs returned using the UNION set operation.

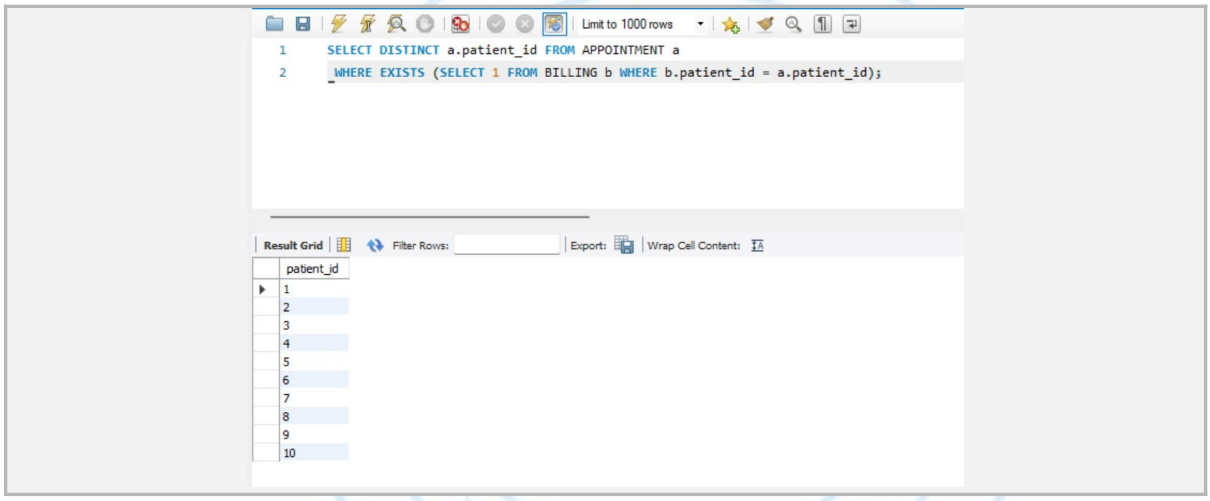


Fig. 7.19: Patients in both appointments and billing using the INTERSECT equivalent.

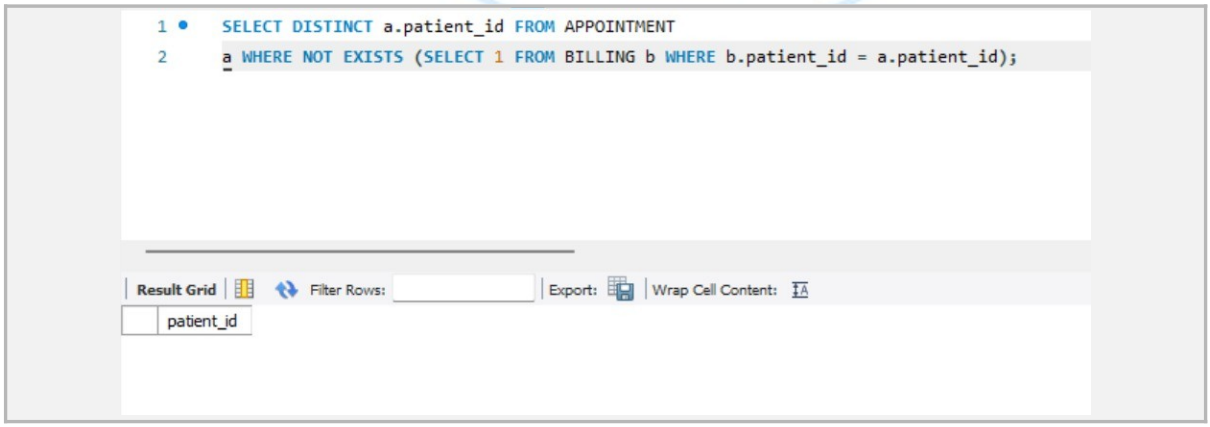


Fig. 7.20: Appointment patients without bills using the EXCEPT equivalent.

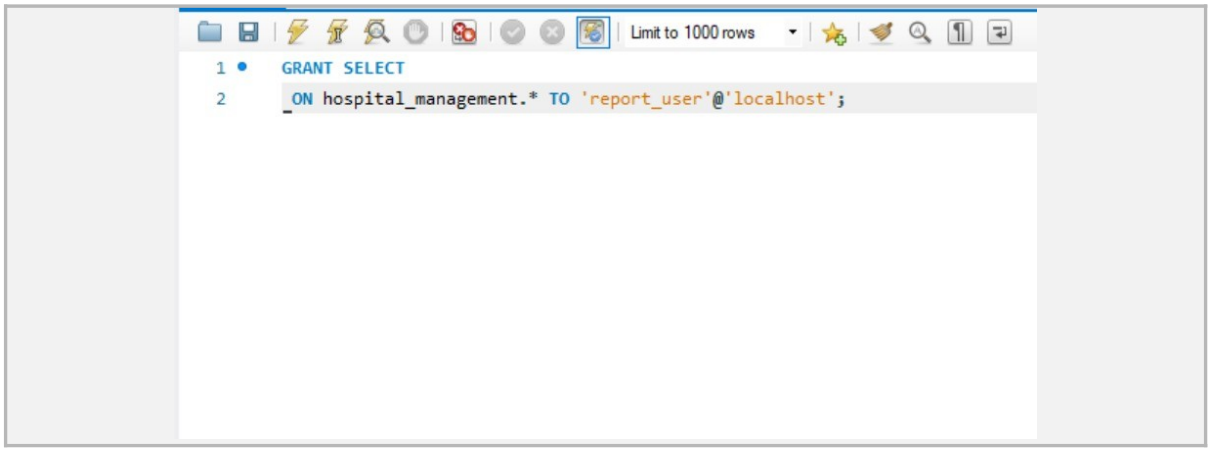


Fig. 7.21: SELECT privilege granted to the reporting user using GRANT.

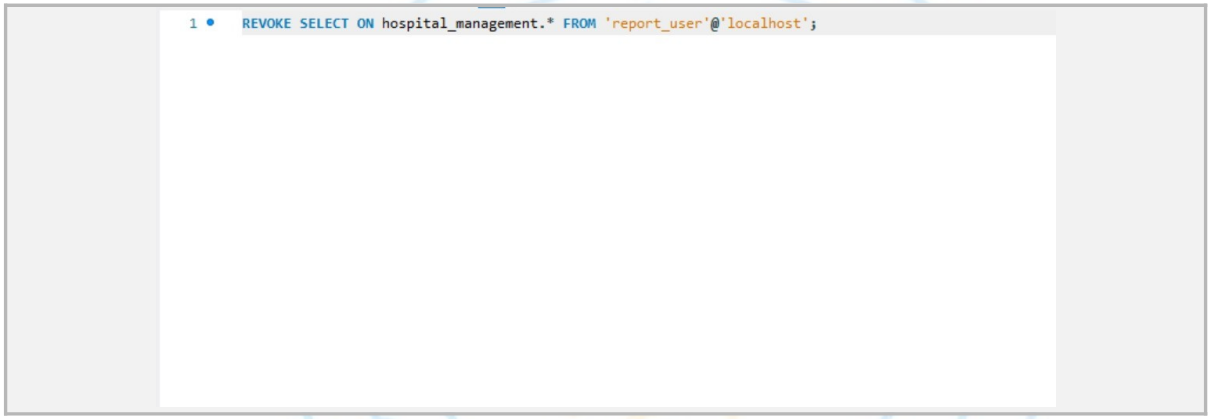


Fig. 7.22: SELECT privilege removed from the reporting user using REVOKE.

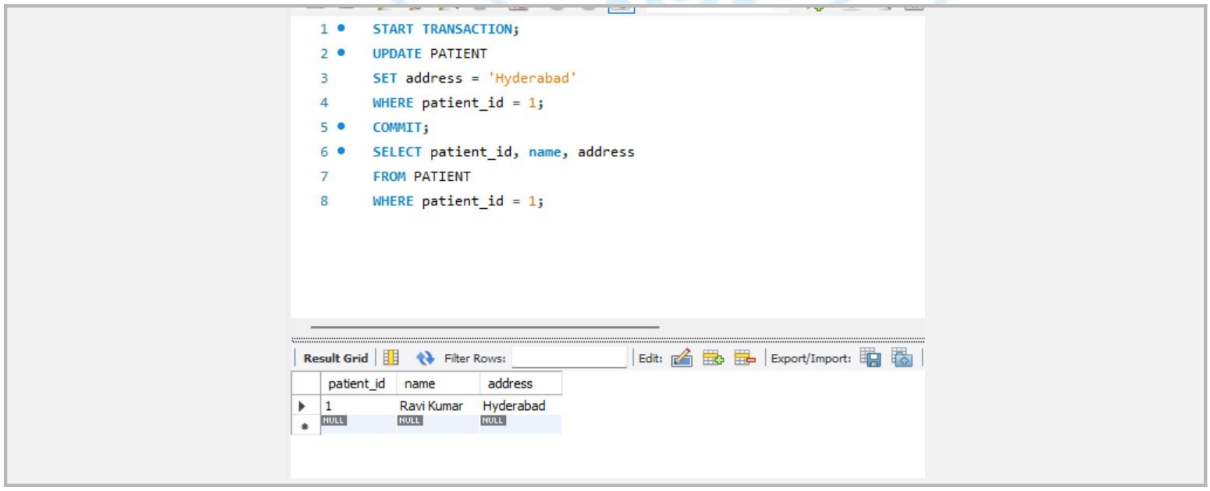


Fig. 7.23: Appointment status updated and saved using COMMIT.

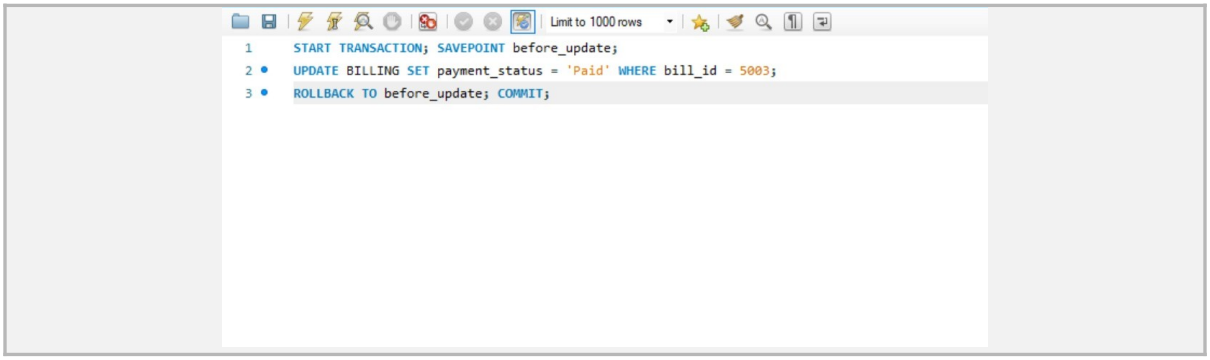


Fig. 7.24: Billing update reverted using SAVEPOINT and ROLLBACK.

CHAPTER 8: TESTING AND DATA VALIDATION

The following test cases verify table creation, sample-data insertion, referential integrity, and query correctness. Results marked Pass have been verified with screenshot evidence (see Fig. 4.1, 4.2 and 4.3). TC3 and TC5 are marked Pending as they still require their own constraint-violation screenshots; only the TC4 scenario (invalid appointment patient) has been captured so far.

Table 8.1: Test Cases and Results

TC No.	Test Case Description	Input / Action	Expected Result	Actual Result	Status
TC1	Database creation	Run SQL/DDL.sql	Database and eight tables are created	Created successfully	Pass
TC2	Sample data insertion	Run SQL/DML.sql	Ten records inserted into each table	Ten sample rows per table	Pass
TC3	Duplicate primary key	Insert PATIENT with patient_id = 1	Primary-key error; row rejected	Expected MySQL duplicate-key error	Pending
TC4	Invalid appointment patient	Insert APPOINTMENT with patient_id = 999	Foreign-key error; row rejected	Expected MySQL foreign-key error	Pass
TC5	Invalid billing patient	Insert BILLING with patient_id = 999	Foreign-key error; row rejected	Expected MySQL foreign-key error	Pending
TC6	Join correctness	Run Q7	Each appointment shows matching patient and doctor	Matches sample data	Pass
TC7	Aggregate correctness	Run Q11 and Q12	Totals and grouped counts are returned	Values computed from billing and patient data	Pass
TC8	Transaction control	Run Q24	Update after savepoint is rolled back	Original payment status retained	Pass

CHAPTER 9: GITHUB REPOSITORY AND TEAM CONTRIBUTION

9.1 Repository Details

GitHub Repository Link: https://github.com/yaswanthreddy1507-bit/DBMS-HOSPITAL-MANAGEMENT-SYSTEM

9.2 Team Responsibilities and Contribution

Table 9.1: Team Responsibilities

S.No	Student Name	Roll Number	Role / Responsibility	Modules Contributed
1	B. Nandha Kishore	25B11AI110	Database design and ER diagram	Patient and Doctor Database design;
2	V. Ram Nikhil	25B11AIC32	Database implementation and SQL development	Appointment and Prescription tables, SQL queries
3	K. Yaswanth Reddy	25B11AI525	Repository maintenance and contribution details to be confirmed	Billing-related SQL; repository maintenance; report preparation
4	G. Mahesh	25B11AI335	Data base testing and project documentation	Ward, Admission and staff tables; Validation and documentation

9.3 Review-wise Progress

Table 9.2: Review-wise Progress Record

Review	Date	Deliverables Presented	Remarks of the Guide
Review I	To be entered	Problem statement, identified entities, ER diagram, relational schema and normalization	Project concept and database design presented for academic review
Review II	To be entered	DDL, DML, constraints, sample data and SQL query preparation	Database prototype and SQL work presented as semester project work
Final Review	To be entered	SQL queries, screenshots, testing, GitHub repository and completed report	To be entered

CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Hospital Management System database was designed and implemented in MySQL using eight related tables: Patient, Doctor, Ward, Staff, Appointment, Billing, Admission, and Prescription. The design uses primary keys and nine foreign-key constraints to maintain links across the hospital workflow. The implementation includes DDL, DML with ten sample records per table, and DQL queries for retrieval, filtering, ordering, distinct values, aggregate calculations, and grouped results. The project meets the stated objectives of centralized data storage, relational integrity, efficient retrieval, and a foundation for future extensions.

10.2 Limitations of the Present Work

The current work is a database-focused prototype. It does not include a front-end interface, authentication or role-based access control, stored procedures, triggers, indexes, automated backups, insurance/pharmacy integrations, or multi-branch support. The repository also requires the remaining advanced SQL queries and execution screenshots to be added before final evaluation.

10.3 Future Enhancements

Future work can add a web or desktop interface, role-based access for receptionists, doctors, administrators, and billing staff, stored procedures for billing and discharge workflows, triggers to maintain available-bed counts and calculated bill totals, indexes for commonly searched columns, validation constraints, backups and recovery procedures, dashboards, and analytics reports.

REFERENCES

The following references supported the database design and MySQL implementation.

1. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, “Database System Concepts”, 8th Edition, McGraw-Hill, ISBN: 978-1260230508.
2. Raghurama Krishnan and Johannes Gehrke, “Database Management Systems”, 4th Edition, McGraw-Hill, ISBN: 978-1260091195.
3. Ramez Elmasri and Shamkant B. Navathe, “Fundamentals of Database Systems”, 7th Edition, Pearson, ISBN: 978-0137504277.
4. MySQL Documentation, MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/refman/8.0/en/.
5. diagrams.net, https://www.diagrams.net/.

APPENDIX A – COMPLETE SQL SCRIPT

Complete source files are available in the GitHub repository: SQL/DDL.sql, SQL/DML.sql, and SQL/DQL.sql. Before submission, append the advanced SQL statements documented in Chapter 5 to SQL/DQL.sql and paste the final combined script here if required by the guide.

```
/* ============================================================
   HOSPITAL MANAGEMENT SYSTEM DATABASE
   COMPLETE SQL SCRIPT
   ============================================================ */

/* ============================================================
   1. DATABASE CREATION
   ============================================================ */

DROP DATABASE IF EXISTS hospital_management;

CREATE DATABASE hospital_management;

USE hospital_management;



/* ============================================================
   2. DDL - TABLE CREATION
   ============================================================ */



/* ------------------------
   2.1 PATIENT TABLE
   ------------------------ */

CREATE TABLE PATIENT (
```

```
    patient_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT,
    gender VARCHAR(10),
    contact_no VARCHAR(15),
    address VARCHAR(100),
    blood_group VARCHAR(5),
    medical_history VARCHAR(200)
);



/* -------------------------
   2.2 DOCTOR TABLE
   ------------------------- */

CREATE TABLE DOCTOR (
    doctor_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    specialization VARCHAR(50),
    department VARCHAR(50),
    contact_no VARCHAR(15),
    availability_status VARCHAR(20)
);



/* -------------------------
   2.3 WARD TABLE
   ------------------------- */

CREATE TABLE WARD (
    ward_id INT PRIMARY KEY,
```

```
    ward_type VARCHAR(30),
    ward_number VARCHAR(10),
    capacity INT,
    available_beds INT
);



/* -------------------------
   2.4 STAFF TABLE
   ------------------------- */

CREATE TABLE STAFF (
    staff_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    role VARCHAR(50),
    department VARCHAR(50),
    contact_no VARCHAR(15),
    shift_timing VARCHAR(30)
);



/* -------------------------
   2.5 APPOINTMENT TABLE
   ------------------------- */

CREATE TABLE APPOINTMENT (
    appointment_id INT PRIMARY KEY,
    patient_id INT,
    doctor_id INT,
    appointment_date DATE,
    appointment_time TIME,
```

```
    status VARCHAR(20),

    FOREIGN KEY (patient_id)
        REFERENCES PATIENT(patient_id),

    FOREIGN KEY (doctor_id)
        REFERENCES DOCTOR(doctor_id)
);



/* -------------------------
   2.6 BILLING TABLE
   ------------------------- */

CREATE TABLE BILLING (
    bill_id INT PRIMARY KEY,
    patient_id INT,
    treatment_amount DECIMAL(10,2),
    room_charges DECIMAL(10,2),
    total_amount DECIMAL(10,2),
    payment_status VARCHAR(20),
    bill_date DATE,

    FOREIGN KEY (patient_id)
        REFERENCES PATIENT(patient_id)
);



/* -------------------------
   2.7 ADMISSION TABLE
   ------------------------- */
```

```
CREATE TABLE ADMISSION (
    admission_id INT PRIMARY KEY,
    patient_id INT,
    ward_id INT,
    attending_doctor_id INT,
    admission_date DATE,
    discharge_date DATE,

    FOREIGN KEY (patient_id)
        REFERENCES PATIENT(patient_id),

    FOREIGN KEY (ward_id)
        REFERENCES WARD(ward_id),

    FOREIGN KEY (attending_doctor_id)
        REFERENCES DOCTOR(doctor_id)
);



/* ------------------------
   2.8 PRESCRIPTION TABLE
   ------------------------ */

CREATE TABLE PRESCRIPTION (
    prescription_id INT PRIMARY KEY,
    appointment_id INT,
    doctor_id INT,
    patient_id INT,
    medicine_name VARCHAR(50),
    dosage VARCHAR(30),
```

```
    duration VARCHAR(30),

    FOREIGN KEY (appointment_id)
        REFERENCES APPOINTMENT(appointment_id),

    FOREIGN KEY (doctor_id)
        REFERENCES DOCTOR(doctor_id),

    FOREIGN KEY (patient_id)
        REFERENCES PATIENT(patient_id)
);



/* ============================================================
   3. DML - SAMPLE DATA INSERTION
   ============================================================ */



/* -------------------------
   3.1 PATIENT DATA
   ------------------------- */

INSERT INTO PATIENT
(patient_id, name, age, gender, contact_no, address,
 blood_group, medical_history)
VALUES
(1, 'Ravi Kumar', 25, 'Male', '9876543210',
 'Hyderabad', 'B+', 'Fever'),

(2, 'Anjali Rao', 32, 'Female', '9876543211',
 'Vijayawada', 'A+', 'Diabetes'),
```

```
(3, 'Suresh Reddy', 45, 'Male', '9876543212',
 'Guntur', 'O+', 'Hypertension'),

(4, 'Priya Sharma', 28, 'Female', '9876543213',
 'Visakhapatnam', 'AB+', 'Asthma'),

(5, 'Arjun Singh', 36, 'Male', '9876543214',
 'Warangal', 'B-', 'None'),

(6, 'Lakshmi Devi', 52, 'Female', '9876543215',
 'Kakinada', 'A-', 'Diabetes'),

(7, 'Kiran Kumar', 41, 'Male', '9876543216',
 'Rajahmundry', 'O-', 'Heart Disease'),

(8, 'Sneha Patel', 23, 'Female', '9876543217',
 'Nellore', 'B+', 'Migraine'),

(9, 'Manoj Rao', 60, 'Male', '9876543218',
 'Tirupati', 'AB-', 'Arthritis'),

(10, 'Divya Reddy', 30, 'Female', '9876543219',
 'Amalapuram', 'O+', 'None');


/* -------------------------
   3.2 DOCTOR DATA
   ------------------------- */

INSERT INTO DOCTOR
```

```
(doctor_id, name, specialization, department,
 contact_no, availability_status)
VALUES
(101, 'Dr. Ramesh', 'Cardiologist',
 'Cardiology', '9000000001', 'Available'),

(102, 'Dr. Suresh', 'Neurologist',
 'Neurology', '9000000002', 'Available'),

(103, 'Dr. Priya', 'Pediatrician',
 'Pediatrics', '9000000003', 'Unavailable'),

(104, 'Dr. Anil', 'Orthopedic',
 'Orthopedics', '9000000004', 'Available'),

(105, 'Dr. Kavya', 'Dermatologist',
 'Dermatology', '9000000005', 'Available'),

(106, 'Dr. Raj', 'General Physician',
 'General Medicine', '9000000006', 'Available'),

(107, 'Dr. Meena', 'Gynecologist',
 'Gynecology', '9000000007', 'Unavailable'),

(108, 'Dr. Vijay', 'ENT Specialist',
 'ENT', '9000000008', 'Available'),

(109, 'Dr. Neha', 'Ophthalmologist',
 'Ophthalmology', '9000000009', 'Available'),

(110, 'Dr. Arun', 'Pulmonologist',
```

```
 'Pulmonology', '9000000010', 'Available');



/* -------------------------
   3.3 WARD DATA
   ------------------------- */

INSERT INTO WARD
(ward_id, ward_type, ward_number, capacity, available_beds)
VALUES
(201, 'General', 'G01', 20, 12),
(202, 'General', 'G02', 20, 15),
(203, 'ICU', 'I01', 10, 4),
(204, 'ICU', 'I02', 10, 6),
(205, 'Private', 'P01', 5, 2),
(206, 'Private', 'P02', 5, 3),
(207, 'Emergency', 'E01', 15, 8),
(208, 'Maternity', 'M01', 12, 7),
(209, 'Pediatric', 'PD01', 10, 5),
(210, 'General', 'G03', 20, 14);



/* -------------------------
   3.4 STAFF DATA
   ------------------------- */

INSERT INTO STAFF
(staff_id, name, role, department,
 contact_no, shift_timing)
VALUES
(601, 'Ramesh Kumar', 'Nurse',
```

```
 'General Medicine', '9100000001', 'Morning'),

(602, 'Sita Devi', 'Nurse',
 'Cardiology', '9100000002', 'Evening'),

(603, 'Mohan Rao', 'Receptionist',
 'Administration', '9100000003', 'Morning'),

(604, 'Geetha Sharma', 'Nurse',
 'ICU', '9100000004', 'Night'),

(605, 'Rajesh Kumar', 'Lab Technician',
 'Laboratory', '9100000005', 'Morning'),

(606, 'Anita Reddy', 'Pharmacist',
 'Pharmacy', '9100000006', 'Evening'),

(607, 'Karthik Singh', 'Nurse',
 'Emergency', '9100000007', 'Night'),

(608, 'Meena Rao', 'Receptionist',
 'Administration', '9100000008', 'Evening'),

(609, 'Vijay Kumar', 'Lab Technician',
 'Laboratory', '9100000009', 'Morning'),

(610, 'Pooja Devi', 'Nurse',
 'Pediatrics', '9100000010', 'Morning');


/* ------------------------
```

```
   3.5 APPOINTMENT DATA
   ------------------------ */

INSERT INTO APPOINTMENT
(appointment_id, patient_id, doctor_id,
 appointment_date, appointment_time, status)
VALUES
(1001, 1, 106, '2026-09-01', '09:00:00', 'Completed'),

(1002, 2, 101, '2026-09-02', '10:00:00', 'Completed'),

(1003, 3, 102, '2026-09-03', '11:00:00', 'Scheduled'),

(1004, 4, 105, '2026-09-04', '09:30:00', 'Completed'),

(1005, 5, 104, '2026-09-05', '12:00:00', 'Scheduled'),

(1006, 6, 106, '2026-09-06', '10:30:00', 'Completed'),

(1007, 7, 101, '2026-09-07', '14:00:00', 'Scheduled'),

(1008, 8, 102, '2026-09-08', '15:00:00', 'Completed'),

(1009, 9, 104, '2026-09-09', '11:30:00', 'Cancelled'),

(1010, 10, 110, '2026-09-10', '16:00:00', 'Scheduled');



/* -------------------------
   3.6 BILLING DATA
   ------------------------ */
```

```
INSERT INTO BILLING
(bill_id, patient_id, treatment_amount,
 room_charges, total_amount, payment_status, bill_date)
VALUES
(5001, 1, 2000.00, 1000.00, 3000.00,
 'Paid', '2026-09-01'),

(5002, 2, 3500.00, 1500.00, 5000.00,
 'Paid', '2026-09-02'),

(5003, 3, 5000.00, 2000.00, 7000.00,
 'Pending', '2026-09-03'),

(5004, 4, 2500.00, 1000.00, 3500.00,
 'Paid', '2026-09-04'),

(5005, 5, 4000.00, 1500.00, 5500.00,
 'Pending', '2026-09-05'),

(5006, 6, 3000.00, 2000.00, 5000.00,
 'Paid', '2026-09-06'),

(5007, 7, 6000.00, 2500.00, 8500.00,
 'Pending', '2026-09-07'),

(5008, 8, 1800.00, 800.00, 2600.00,
 'Paid', '2026-09-08'),

(5009, 9, 4500.00, 2000.00, 6500.00,
 'Pending', '2026-09-09'),
```

```
(5010, 10, 2200.00, 1000.00, 3200.00,
 'Paid', '2026-09-10');



/* -------------------------
   3.7 ADMISSION DATA
   ------------------------- */

INSERT INTO ADMISSION
(admission_id, patient_id, ward_id,
 attending_doctor_id, admission_date, discharge_date)
VALUES
(3001, 1, 201, 106,
 '2026-09-01', '2026-09-03'),

(3002, 2, 202, 101,
 '2026-09-02', '2026-09-05'),

(3003, 3, 203, 101,
 '2026-09-03', NULL),

(3004, 4, 205, 105,
 '2026-09-04', '2026-09-06'),

(3005, 5, 206, 104,
 '2026-09-05', '2026-09-08'),

(3006, 6, 201, 106,
 '2026-09-06', '2026-09-10'),
```

```
(3007, 7, 204, 101,
 '2026-09-07', NULL),

(3008, 8, 209, 102,
 '2026-09-08', '2026-09-09'),

(3009, 9, 202, 104,
 '2026-09-09', NULL),

(3010, 10, 207, 110,
 '2026-09-10', '2026-09-12');



/* -------------------------
   3.8 PRESCRIPTION DATA
   ------------------------- */

INSERT INTO PRESCRIPTION
(prescription_id, appointment_id, doctor_id,
 patient_id, medicine_name, dosage, duration)
VALUES
(4001, 1001, 106, 1,
 'Paracetamol', '500 mg', '5 days'),

(4002, 1002, 101, 2,
 'Metformin', '500 mg', '30 days'),

(4003, 1003, 102, 3,
 'Aspirin', '75 mg', '15 days'),

(4004, 1004, 105, 4,
```

```
 'Cetirizine', '10 mg', '7 days'),

(4005, 1005, 104, 5,
 'Ibuprofen', '400 mg', '5 days'),

(4006, 1006, 106, 6,
 'Metformin', '500 mg', '30 days'),

(4007, 1007, 101, 7,
 'Atorvastatin', '20 mg', '30 days'),

(4008, 1008, 102, 8,
 'Sumatriptan', '50 mg', '5 days'),

(4009, 1009, 104, 9,
 'Calcium', '500 mg', '20 days'),

(4010, 1010, 110, 10,
 'Montelukast', '10 mg', '10 days');



/* ============================================================
   4. DQL - BASIC RETRIEVAL QUERIES
   ============================================================ */



/* Display all patient records */

SELECT * FROM PATIENT;
```

```
/* Display all doctor records */

SELECT * FROM DOCTOR;


/* Display all ward records */

SELECT * FROM WARD;


/* Display all staff records */

SELECT * FROM STAFF;


/* Display all appointment records */

SELECT * FROM APPOINTMENT;


/* Display all billing records */

SELECT * FROM BILLING;


/* Display all admission records */

SELECT * FROM ADMISSION;


/* Display all prescription records */
```

```
SELECT * FROM PRESCRIPTION;


/* Patients older than 30 and female */

SELECT *
FROM PATIENT
WHERE age > 30
AND gender = 'Female';


/* Patients younger than 30 OR older than 50 */

SELECT *
FROM PATIENT
WHERE age < 30
OR age > 50;


/* Doctors who are available */

SELECT *
FROM DOCTOR
WHERE availability_status = 'Available';


/* Display patients from youngest to oldest */

SELECT *
FROM PATIENT
```

```
ORDER BY age ASC;



/* Display different blood groups */

SELECT DISTINCT blood_group
FROM PATIENT;



/* ============================================================
   5. JOIN OPERATIONS
   ============================================================ */



/* INNER JOIN */

SELECT
    a.appointment_id,
    p.name AS patient,
    d.name AS doctor,
    a.appointment_date
FROM APPOINTMENT a
INNER JOIN PATIENT p
    ON a.patient_id = p.patient_id
INNER JOIN DOCTOR d
    ON a.doctor_id = d.doctor_id;



/* LEFT JOIN */

SELECT
```

```
    p.patient_id,
    p.name,
    a.appointment_id,
    a.status
FROM PATIENT p
LEFT JOIN APPOINTMENT a
    ON p.patient_id = a.patient_id;



/* RIGHT JOIN */

SELECT
    d.doctor_id,
    d.name,
    a.appointment_id,
    a.status
FROM APPOINTMENT a
RIGHT JOIN DOCTOR d
    ON a.doctor_id = d.doctor_id;



/* FULL OUTER JOIN simulation using UNION */

SELECT
    p.patient_id,
    p.name,
    b.bill_id
FROM PATIENT p
LEFT JOIN BILLING b
    ON p.patient_id = b.patient_id
```

```
UNION

SELECT
   p.patient_id,
   p.name,
   b.bill_id
FROM PATIENT p
RIGHT JOIN BILLING b
   ON p.patient_id = b.patient_id;



/* ============================================================
  6. AGGREGATE FUNCTIONS, GROUP BY AND HAVING
  ============================================================ */



/* Total treatment amount */

SELECT
   SUM(treatment_amount) AS total_treatment
FROM BILLING;



/* Count patients by gender */

SELECT
   gender,
   COUNT(*) AS total_patients
FROM PATIENT
GROUP BY gender;
```

```
/* Average bill by payment status */

SELECT
   payment_status,
   AVG(total_amount) AS average_bill
FROM BILLING
GROUP BY payment_status
HAVING AVG(total_amount) > 4000;



/* ============================================================
   7. NESTED AND CORRELATED SUBQUERIES
   ============================================================ */



/* Patients older than average age */

SELECT
   name,
   age
FROM PATIENT
WHERE age > (
   SELECT AVG(age)
   FROM PATIENT
);



/* Doctors who have completed appointments */

SELECT
```

```
    d.name
FROM DOCTOR d
WHERE EXISTS (
    SELECT 1
    FROM APPOINTMENT a
    WHERE a.doctor_id = d.doctor_id
    AND a.status = 'Completed'
);



/* ============================================================
   8. VIEWS
   ============================================================ */



/* View of available doctors */

CREATE VIEW available_doctors AS
SELECT
    doctor_id,
    name,
    specialization,
    department
FROM DOCTOR
WHERE availability_status = 'Available';



/* View of patient billing summary */

CREATE VIEW patient_bill_summary AS
SELECT
```

```
    p.patient_id,
    p.name,
    SUM(b.total_amount) AS total_billed
FROM PATIENT p
JOIN BILLING b
    ON p.patient_id = b.patient_id
GROUP BY
    p.patient_id,
    p.name;



/* ============================================================
   9. RELATIONAL SET OPERATIONS
   ============================================================ */



/* UNION */

SELECT patient_id
FROM APPOINTMENT

UNION

SELECT patient_id
FROM BILLING;



/* INTERSECT equivalent using EXISTS */

SELECT DISTINCT
    a.patient_id
```

```
FROM APPOINTMENT a
WHERE EXISTS (
   SELECT 1
   FROM BILLING b
   WHERE b.patient_id = a.patient_id
);



/* EXCEPT equivalent using NOT EXISTS */

SELECT DISTINCT
   a.patient_id
FROM APPOINTMENT a
WHERE NOT EXISTS (
   SELECT 1
   FROM BILLING b
   WHERE b.patient_id = a.patient_id
);



/* ============================================================
  10. DCL - DATA CONTROL LANGUAGE
  ============================================================ */



/* Grant SELECT privilege */

GRANT SELECT
ON hospital_management.*
TO 'report_user'@'localhost';
```

```
/* Revoke SELECT privilege */

REVOKE SELECT
ON hospital_management.*
FROM 'report_user'@'localhost';



/* ============================================================
   11. TCL - TRANSACTION CONTROL LANGUAGE
   ============================================================ */



/* COMMIT */

START TRANSACTION;

UPDATE APPOINTMENT
SET status = 'Completed'
WHERE appointment_id = 1003;

COMMIT;



/* SAVEPOINT and ROLLBACK */

START TRANSACTION;

SAVEPOINT before_update;

UPDATE BILLING
```

```
SET payment_status = 'Paid'
WHERE bill_id = 5003;

ROLLBACK TO before_update;

COMMIT;



/* ============================================================
   END OF COMPLETE SQL SCRIPT
   HOSPITAL MANAGEMENT SYSTEM
   ============================================================ */
```